University of Trento

Statistical Models

Notes by Lorenzo Di Berardino

Last update: 25/5/2026

February 2026—May 2026

Contents

1 Introduction

1.1 What we are going to talk about

In this course we will talk about *statistical learning*, which includes both old and modern approaches:

1. *Linear regression* (Gauss, 1800s), which provides a quantitative response.
2. *Linear Discriminant Analysis* (LDA) (Fisher, 1936), which provides a qualitative response. It can be extended to *Quadratic Discriminant Analysis* (QDA) and applied to Naive Bayes classifiers.
3. *Logistic Regression* (1940s)
4. *Generalized Linear Models* (GLMs) (1970s)
5. *Classification and Regression Trees* (CRTs) (1980s), which are computer-intensive methods and can be extended to *Random Forests* and *Boosting*.
6. *Machine Learning* (1990s), which includes Support Vector Machines, Neural Networks, Deep Learning, Unsupervised Learning (Clustering and PCA), etc.

1.2 What is Statistical Learning?

Example. Suppose we have a dataset containing wage data related to workers in the US. Our goal is to predict the wage from a number of factors (predictors), such as education, age, etc. ◇

We could formalize the problem as follows:

- Let Y be the response (dependent variable),
- Let $\mathbf{X}$ be the predictor (independent variable), a vector composed of p predictors, *i.e.*:

$$\mathbf{X} = (X_1, X_2, \ldots, X_p)^\intercal. \tag{1.1}$$

- Then, $Y = f(\mathbf{X}) + \varepsilon$ is our prediction. The f is a fixed but unknown function of $X_1, \ldots, X_p$ (systematic component) and ε is a stochastic component such that:

$$\mathrm{E}[\varepsilon] = 0 \quad \text{and} \quad \varepsilon \perp \mathbf{X}. \tag{1.2}$$

The perpendicular symbol means that ε is independent of $\mathbf{X}$, so the error does not depend on the predictors.

The estimation of f is important for two reasons:

Prediction We want to predict Y when we only have observations of $\mathbf{X}$. Given $\hat{f}$ estimate of f, the prediction of Y when $\mathbf{X} = \mathbf{x}$ is $\hat{Y} = \hat{f}(\mathbf{x})$. If this is the only goal, then $\hat{f}$ can also be a black-box,

i.e., we are not interested in the relationship between Y and $\mathbf{X}$, but only in the accuracy of the prediction.

The accuracy of $\hat{f}$ as a prediction for y can be described by:

$$\begin{aligned}
\mathrm{E}[(Y - \hat{f}(\mathbf{x}))^2 \mid \mathbf{X} = \mathbf{x}] &= \mathrm{E}[(f(\mathbf{x}) + \varepsilon - \hat{f}(\mathbf{x}))^2 \mid \mathbf{X} = \mathbf{x}] \\
&= \mathrm{E}[(f(\mathbf{x}) - \hat{f}(\mathbf{x}) + \varepsilon)^2 \mid \mathbf{X} = \mathbf{x}] \\
&= \mathrm{E}[(f(\mathbf{x}) - \hat{f}(\mathbf{x}))^2 \mid \mathbf{X} = \mathbf{x}] + \mathrm{E}[\varepsilon^2] \\
&\quad + 2\,\mathrm{E}[(f(\mathbf{x}) - \hat{f}(\mathbf{x}))\varepsilon \mid \mathbf{X} = \mathbf{x}] \\
&= (f(\mathbf{x}) - \hat{f}(\mathbf{x}))^2 + \mathrm{Var}(\varepsilon) + 2(f(\mathbf{x}) - \hat{f}(\mathbf{x}))\underbrace{\mathrm{E}[\varepsilon]}_{0} \\
&= (f(\mathbf{x}) - \hat{f}(\mathbf{x}))^2 + \mathrm{Var}(\varepsilon).
\end{aligned} \tag{1.3}$$

The variance of ε is called *irreducible error*, since it cannot be reduced by any method, while the first term is called *reducible error*, since it can be reduced by choosing a better $\hat{f}$.

Inference We want to answer to different questions:

1. Which predictors are associated with the response?
2. What is the relationship between Y and each predictor?
3. How do we estimate f?

Broadly speaking, there are two types of methods to estimate f:

- *Parametric approaches*: we make an assumption about the functional form of f, which depends on parameters, *e.g.*:

  $$f(\mathbf{X}) = \beta_0 + \beta_1 X_1 + \cdots + \beta_p X_p. \tag{1.4}$$

  So, parameters are unknown and statistical learning becomes parameter estimation of:

  $$\hat{\beta}_0, \hat{\beta}_1, \ldots, \hat{\beta}_p. \tag{1.5}$$

  Once we have them, we can make estimations.

  A disadvantage of this method is that f may be far from the parametric family we have chosen, while an advantage is the possibility of interpretation of the model (inference).

  Note. Linear models are *linear in the parameters*, so:

  $$f(x_1) = \beta_0 + \beta_1 x_1 + \beta_2 x_1^2 + \cdots + \beta_p x_1^p \tag{1.6}$$

  is allowed, while, for example:

  $$f(x_2) = \beta_0 e^{\beta_1 x_2} \tag{1.7}$$

  is not linear in the parameters, so it is not allowed. ○

- *Non-parametric approaches*: no explicit assumptions about the functional form of f are made. They estimate f by getting as close as possible to the data, trying not to be too rough or wiggly (overfitting). These methods are more flexible, but, as said, they have greater potential for overfitting.

1.3 Bias-Variance Trade-off

The bias-variance tradeoff is a fundamental concept in statistical learning, which describes the relationship between the bias and variance of an estimator. Suppose that we compute the mean squared error (MSE) of an estimator $\hat{f}$ at a point $\mathbf{x}$:

$$
\begin{aligned}
\mathrm{E}[(Y - \hat{f}(\mathbf{X}))^2 \mid \mathbf{X} = \mathbf{x}] &= \mathrm{E}[(f(\mathbf{X}) + \varepsilon - \hat{f}(\mathbf{X}) + \mathrm{E}[\hat{f}(\mathbf{X})] - \mathrm{E}[\hat{f}(\mathbf{X})])^2 \mid \mathbf{X} = \mathbf{x}] \\
&= \mathrm{E}[\underbrace{(f(\mathbf{X}) - \mathrm{E}[\hat{f}(\mathbf{X})]}_{a} + \underbrace{\varepsilon}_{b} + \underbrace{\mathrm{E}[\hat{f}(\mathbf{X})] - \hat{f}(\mathbf{X}))}_{c}{}^2 \mid \mathbf{X} = \mathbf{x}] \\
&= \mathrm{E}[(f(\mathbf{X}) - \mathrm{E}[\hat{f}(\mathbf{X})])^2 \mid \mathbf{X} = \mathbf{x}] + \mathrm{E}[\varepsilon^2] \\
&\quad + \mathrm{E}[(\hat{f}(\mathbf{X}) - \mathrm{E}[\hat{f}(\mathbf{X})])^2 \mid \mathbf{X} = \mathbf{x}] \\
&\quad + 2\,\mathrm{E}[(f(\mathbf{X}) - \mathrm{E}[\hat{f}(\mathbf{X})])\varepsilon \mid \mathbf{X} = \mathbf{x}] \\
&\quad + 2\,\mathrm{E}[(f(\mathbf{X}) - \mathrm{E}[\hat{f}(\mathbf{X})])(\mathrm{E}[\hat{f}(\mathbf{X})] - \hat{f}(\mathbf{X})) \mid \mathbf{X} = \mathbf{x}] \\
&\quad + 2\,\mathrm{E}[\varepsilon(E[\hat{f}(\mathbf{X})] - \hat{f}(\mathbf{X})) \mid \mathbf{X} = \mathbf{x}].
\end{aligned} \tag{1.8}
$$

Note. The term $\mathrm{E}[\hat{f}(\mathbf{X})]$ is the expectation taken over $n \to \infty$ datasets. ○

We can check that all the terms except the first three are zero:

- $2\,\mathrm{E}[(f(\mathbf{X}) - \mathrm{E}[\hat{f}(\mathbf{X})])\varepsilon \mid \mathbf{X} = \mathbf{x}](f(\mathbf{X}) - \mathrm{E}[\hat{f}(\mathbf{X})])\,\mathrm{E}[\varepsilon] = 0$ since $\mathrm{E}[\varepsilon] = 0$,
- $2\,\mathrm{E}[(f(\mathbf{X}) - \mathrm{E}[\hat{f}(\mathbf{X})])(\mathrm{E}[\hat{f}(\mathbf{X})] - \hat{f}(\mathbf{X})) \mid \mathbf{X} = \mathbf{x}] = 0$ since $\mathrm{E}[\mathrm{E}[Z] - Z] = 0$ for any random variable Z,
- $2\,\mathrm{E}[\varepsilon(E[\hat{f}(\mathbf{X})] - \hat{f}(\mathbf{X})) \mid \mathbf{X} = \mathbf{x}] = 0$ since $\mathrm{E}[\varepsilon] = 0$ and $\mathrm{E}[E[\hat{f}(\mathbf{X})] - \hat{f}(\mathbf{X})] = 0$.

Therefore, we have:

$$
\underbrace{(f(\mathbf{x}) - \mathrm{E}[\hat{f}(\mathbf{x})])^2}_{\text{Bias}} + \mathrm{Var}(\varepsilon) + \mathrm{Var}(\hat{f}(\mathbf{x})). \tag{1.9}
$$

Simple models tend to have high bias and low variance, meaning that on new data, $\hat{f}$ does not change much. With a n-degree polynomial we can always pass through $n + 1$ points, so it has low bias and high variance. However, the resulting curve will be wiggly. High degree polynomials (complex models) are more flexible but they are the opposite of simple models, since they have low bias and high variance. The accuracy of the model depends on the a, b and c terms of the Equation (1.8), but b is not reducible, while a and c are inversely proportional. In conclusion, for a given dataset, we need to find a good trade-off between bias and variance.

2 Supervised learning

In this part we will focus on supervised learning: we are given both $\mathbf{x}$ and y and we use this information to learn the relationship between the two. Supervised methods can be broadly divided into two types:

1. *Regression methods* if y is quantitative (numerical).
2. *Classification methods* if y is qualitative (categorical).

No assumptions are made on $\mathbf{x}$, the vector of predictors. The objective is, given observations $(\mathbf{x}_i, y_i)$, $i = 1 \dots n$, to find a rule $\hat{f}$ (regression function/classifier) that allows to predict y from $\mathbf{x}$, *i.e.*:

$$\hat{y} = \hat{f}(\mathbf{x}). \tag{2.1}$$

Here, $\hat{f}$ is an approximation of some true f. Statistical Decision Theory formalizes the type of relationship that we would like to learn.

2.1 Regression methods

Squared Error Loss (SEL) is a method for estimating the relationship between a response variable y and one or more predictor variables $\mathbf{x}$. It is mathematically defined as:

$$\mathrm{L}(Y, f(\mathbf{X})) = (Y - f(\mathbf{X}))^2. \tag{2.2}$$

This loss function is used in the Expected Prediction Error (EPE):

$$\mathrm{EPE}(f) = \mathrm{E}_{\mathbf{X},Y}[\mathrm{L}(Y, f(\mathbf{X}))] = \int (y - f(\mathbf{x}))^2 g(\mathbf{x}, y)\, \mathrm{d}\mathbf{x}\, \mathrm{d}y. \tag{2.3}$$

Then, we choose the f that minimizes the Expected Prediction Error (EPE).

Theorem 1. *The quantity* $\mathrm{E}[(Y - f(\mathbf{X}))^2]$ *is minimized by* $f(\mathbf{x}) = \mathrm{E}[Y \mid \mathbf{X} = \mathbf{x}]$.

Proof. We can write:

$$\begin{aligned} \mathrm{E}_{\mathbf{X},Y}[(Y - f(\mathbf{X}))^2] &= \int (y - f(\mathbf{x}))^2 g(\mathbf{x}, y)\, \mathrm{d}\mathbf{x}\, \mathrm{d}y \\ &= \int \underbrace{(y - f(\mathbf{x}))^2 g_{Y,\mathbf{X}}(y \mid \mathbf{x})\, \mathrm{d}y}_{\mathrm{E}_{Y\mid\mathbf{X}}[(Y - f(\mathbf{X}))^2]} g_{\mathbf{X}}(\mathbf{x})\, \mathrm{d}\mathbf{x} \\ &= \mathrm{E}_{\mathbf{X}}[\mathrm{E}_{Y\mid\mathbf{X}}[(Y - f(\mathbf{X}))^2]]. \end{aligned} \tag{2.4}$$

The last equality follows from the *tower rule*.

Now, let's consider a fixed $\mathbf{x}$ and the inner expectation as $\mathrm{E}_Y[(Y-a)^2]$ where a is a constant. We ask ourselves: what's the value of a that minimizes this quantity? We can write:

$$\begin{aligned}\mathrm{E}[(Y-a)^2] &= \mathrm{E}[Y^2 - 2aY + a^2] \\ &= \mathrm{E}[Y^2] - 2a\,\mathrm{E}[Y] + a^2.\end{aligned} \tag{2.5}$$

To optimize this quantity, we can take the derivative with respect to a and set it to zero:

$$\begin{aligned}\frac{\mathrm{d}}{\mathrm{d}a}\mathrm{E}[(Y-a)^2] &= -2\,\mathrm{E}[Y] + 2a = 0 \\ a^* &= \mathrm{E}[Y].\end{aligned} \tag{2.6}$$

Therefore, $f(\mathbf{x}) = \mathrm{E}[Y \mid \mathbf{X} = \mathbf{x}]$ is the function that minimizes $\mathrm{E}_{Y\mid\mathbf{X}}[(Y - f(\mathbf{X}))^2]$. □

Note. It holds that:

$$Y = f(\mathbf{X}) + \varepsilon \iff \mathrm{E}[Y \mid \mathbf{X} = \mathbf{x}] = f(\mathbf{x}). \tag{2.7}$$ ○

It is worth noting that other choices of the loss function L lead to different optimal regression functions. For example, if we choose $\mathrm{L}(Y, f(\mathbf{X})) = \left|Y - f(\mathbf{X})\right|$, then the optimal regression function is the conditional median of Y given $\mathbf{X}$:

$$f(\mathbf{x}) = \mathrm{Median}(Y \mid \mathbf{X} = \mathbf{x}). \tag{2.8}$$

The use of this loss leads to the so called *Least Absolute Deviation Regression*.

Observation. If you put $a^* = \mathrm{E}[Y]$ in the expression $\mathrm{E}_Y[(Y-a)^2]$, you get the variance of Y:

$$\mathrm{E}[(Y-a^*)^2] = \mathrm{E}[(Y - \mathrm{E}[Y])^2] = \mathrm{Var}(Y). \tag{2.9}$$ △

Under the squared error loss, different regression methods make different assumptions about the function $f(\mathbf{x}) = \mathrm{E}[Y \mid \mathbf{X} = \mathbf{x}]$. For example:

1. *Linear regression* (parametric), makes three assumptions on Y:

   $$Y \mid \mathbf{X} = \mathbf{x} \sim \mathcal{N}(\mathbf{x}^\intercal \beta, \sigma^2), \tag{2.10}$$

   i.e., the normality of Y given $\mathbf{X}$, the linearity of the mean over parameters β and the homoscedasticity of the variance σ^2.

   The function $f(\mathbf{x})$ can be written as:

   $$f(\mathbf{x}) = \mathrm{E}[Y \mid \mathbf{X} = \mathbf{x}] = \beta_0 + \beta_1 x_1 + \ldots + \beta_p x_p. \tag{2.11}$$

2. *k-nearest neighbors* (non-parametric), makes no assumptions on Y and $f(\mathbf{x})$ is estimated as the average of the k nearest neighbors of $\mathbf{x}$:

   $$\hat{f}(\mathbf{x}) = \mathrm{Average}(y_i \mid \mathbf{x}_i \in N_k(\mathbf{x})), \tag{2.12}$$

   where $N_k(\mathbf{x})$ is the set of the k closest observations to $\mathbf{x}$. This simple approach can be extended to kernel-based methods.

2.2 Classification methods

Let Y be a categorical variable that can take K different values (categories). A *classifier* assigns a class to an object $\mathbf{x}$, to produce $\mathrm{Y}(\mathbf{x})$. A loss function is now defined as a $K \times K$ table. In the binary case we obtain a 0-1 loss function:

$$\mathrm{L}(Y, f(\mathbf{X})) = \begin{cases} 0 & \text{if } Y = f(\mathbf{X}) \\ 1 & \text{if } Y \neq f(\mathbf{X}) \end{cases} \tag{2.13}$$

	$\hat{Y} = 0$	$\hat{Y} = 1$
$Y = 0$	0	1
$Y = 1$	1	0

The expected 0-1 loss is computed as:

$$\mathrm{E}[\mathrm{L}(Y, \mathrm{Y}(\mathbf{X}))] = \mathrm{E}_{\mathbf{X}}[\mathrm{E}_{Y \mid \mathbf{X}}[\mathrm{L}(Y, \mathrm{Y}(\mathbf{X}))]]. \tag{2.14}$$

Let's consider a fixed $\mathbf{x}$. Given $\mathbf{x}$, if it is predicted to class 0, *i.e.*, $\mathrm{Y}(\mathbf{x}) = 0$, then the expected loss is:

$$\begin{aligned} \mathrm{E}_Y[\mathrm{L}(Y, 0) \mid \mathbf{x}] &= \underbrace{\mathrm{L}(0,0)}_{0} \mathrm{P}(0 \mid \mathbf{x}) + \underbrace{\mathrm{L}(1,0)}_{1} \mathrm{P}(1 \mid \mathbf{x}) \\ &= \mathrm{P}(1 \mid \mathbf{x}). \end{aligned} \tag{2.15}$$

On the other hand, if it is predicted to class 1:

$$\begin{aligned} \mathrm{E}_Y[\mathrm{L}(Y, 1) \mid \mathbf{x}] &= \underbrace{\mathrm{L}(0,1)}_{1} \mathrm{P}(0 \mid \mathbf{x}) + \underbrace{\mathrm{L}(1,1)}_{0} \mathrm{P}(1 \mid \mathbf{x}) \\ &= \mathrm{P}(0 \mid \mathbf{x}). \end{aligned} \tag{2.16}$$

Therefore, the optimal classifier under a 0-1 loss assigns $\mathbf{x}$ to the class 1 if $\mathrm{P}(0 \mid \mathbf{x}) \leqslant \mathrm{P}(1 \mid \mathbf{x})$ (it has lower loss). This translates into:

$$\mathrm{P}(0 \mid \mathbf{x}) \leqslant \mathrm{P}(1 \mid \mathbf{x}) \iff 1 - \mathrm{P}(1 \mid \mathbf{x}) \leqslant \mathrm{P}(1 \mid \mathbf{x}) \iff \mathrm{P}(1 \mid \mathbf{x}) \geqslant 0.5. \tag{2.17}$$

In general, with more than two classes, the Bayes Classifier assigns $\mathbf{x}$ to the class:

$$j = \underset{i \in \text{Classes}}{\operatorname{argmax}} \mathrm{P}(i \mid \mathbf{x}). \tag{2.18}$$

2.2.1 Bayes Decision Boundary

A *Bayes Decision Boundary* is the set of points $\mathbf{x}$ in the space of predictors where $\mathrm{P}(1 \mid \mathbf{x}) = 0.5$ and is represented by a $(p-1)$-d manifold in a p-dimensional space. Our goal is to approximate the surface with a classifier $\hat{\mathrm{Y}}(\mathbf{x})$ that is as close as possible to the Bayes Classifier $\mathrm{Y}(\mathbf{x})$.

There is a similar concept with the irreducible error in regression but with Bayes Classifier which is called *Bayes Error Rate* and is defined as:

$$\mathrm{BER} = 1 - \mathrm{E}_{\mathbf{X}}[\max_{j} \mathrm{P}(j \mid \mathbf{X})]. \tag{2.19}$$

The inner maximization gives the predicted class for a given $\mathbf{X}$ and the expectation gives the average error across all possible $\mathbf{X}$ (in fact, we are removing from the certainty, 1, the confidence of the prediction). In an ideal case, in which in every $\mathbf{x}$ corresponds to observations of a single class, the Bayes Error Rate is 0. In a more realistic case, in which there are some values of $\mathbf{x}$ for which there are observations of both classes, the Bayes Error Rate is greater than 0. Bayes Error Rate relates to irreducible error because no matter how good our classifier is, there might be points in which two classifications collide and observations cannot be separated.

2.3 Assessing model accuracy

A criterion to choose the loss function is to determine how the model performs on new data. We distinguish between two settings:

1. *Regression setting*:

$$\text{MSE} = \frac{1}{m}\sum_{i=1}^{m}\left(y_i^{(t)} - \hat{f}(\mathbf{x}_i^{(t)})\right)^2, \tag{2.20}$$

   where $(x_i^{(t)}, y_i^{(t)})$, $i = 1, \ldots, m$ are testing data points.

2. *Classification setting:* given a $\hat{\mathrm{P}}(1 \mid \mathbf{x})$ classifier, we can calculate the *confusion matrix*:

	$\hat{Y} = 0$	$\hat{Y} = 1$
$Y = 0$	TN	FP
$Y = 1$	FN	TP

(2.21)

   From this, we can compute different statistics:

   - *True Positive Rate*:

$$\text{TPR} = \frac{\text{TP}}{\text{TP} + \text{FN}}, \tag{2.22}$$

   - *False Positive Rate*:

$$\text{FPR} = \frac{\text{FP}}{\text{FP} + \text{TN}}, \tag{2.23}$$

   and many other, such as *False Discovery Rate*, *Precision*, *Recall*, etc. Another important statistic is the *Error Rate*:

$$\text{ER} = \frac{\text{FP} + \text{FN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}. \tag{2.24}$$

The 0-1 loss is the simplest loss as it treats every misclassification the same, but there can be differences in loss costs based on the type of error (e.g., false positives vs. false negatives). The 0-1 loss can be generalized to a cost-sensitive loss function:

	$\hat{Y} = 0$	$\hat{Y} = 1$
$Y = 0$	0	c_0
$Y = 1$	c_1	0

(2.25)

with $c_0 \neq c_1$. If $c_0 = c_1$, then we are back to the 0-1 loss. Using the same derivations as before, we get:

$$\begin{aligned} \mathrm{E}[\mathrm{L}(Y, 0) \mid \mathbf{x}] &= c_1 \, \mathrm{P}(1 \mid \mathbf{x}), \\ \mathrm{E}[\mathrm{L}(Y, 1) \mid \mathbf{x}] &= c_0 \, \mathrm{P}(0 \mid \mathbf{x}). \end{aligned} \tag{2.26}$$

Therefore, the Bayes Classifier assigns $\mathbf{x}$ to class 1 if $c_0 \, \mathrm{P}(0 \mid \mathbf{x}) \leqslant c_1 \, \mathrm{P}(1 \mid \mathbf{x})$, which leads to:

$$\mathrm{P}(1 \mid \mathbf{x}) \geqslant \frac{c_0}{c_0 + c_1}. \tag{2.27}$$

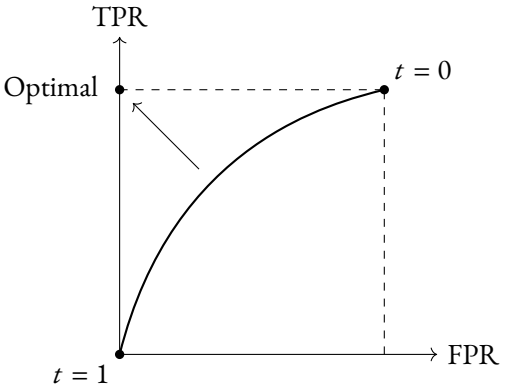

FIGURE 2.1 ROC *curve for two classifiers.*

Observation. If $c_0 = c_1$, then $\frac{c_0}{c_0+c_1} = \frac{1}{2}$, which is the threshold we had before with the 0-1 loss. △

If we don't know where to put the threshold to distinguish between classes, we can plot the Receiver Operating Characteristic (ROC) curve (FIGURE 2.1), which, for varying threshold values, shows the relation between TPR and FPR. Each point on the curve is associated with a specific pair of TPR and FPR values and is derived from a different threshold $t \in [0, 1]$. Having the curve, the closer we get to the optimal point (TPR = 1 and FPR = 0), the better the classifier is. In general, one choses the classifier with the largest Area Under the Curve (AUC) or with a curve that is the closes to the optimal point for the range of values of the threshold (c_0) that one is interested in.

In a regression setting, under a squared error loss, we estimate $\mathrm{E}[Y \mid \mathbf{X} = \mathbf{x}]$, for example, through:

1. *Kernel-based methods*:

$$\widehat{\mathrm{E}[Y \mid \mathbf{X} = \mathbf{x}]} = \mathrm{Average}(y_i \mid \mathbf{x}_i \in N_k(\mathbf{x})). \tag{2.28}$$

2. *Linear regression*:

$$\widehat{\mathrm{E}[Y \mid \mathbf{X} = \mathbf{x}]} = \hat{\beta}_0 + \hat{\beta}_1 x_1 + \cdots + \hat{\beta}_p x_p = \mathbf{x}^\intercal \hat{\boldsymbol{\beta}}. \tag{2.29}$$

In a classification setting, under a 0-1 loss, we estimate $\mathrm{P}(1 \mid \mathbf{X} = \mathbf{x})$, for example, through:

1. *k-nearest neighbors classifier* (non-parametric).
2. *Logistic regression* (parametric).

Those methods will provide an estimate of $\mathrm{P}(1 \mid \mathbf{X} = \mathbf{x})$, namely $\widehat{\mathrm{P}(1 \mid \mathbf{X} = \mathbf{x})}$, or more generally:

$$\mathrm{P}(Y = j \mid \mathbf{X} = \mathbf{x}), \quad j = 1, \ldots, C, \tag{2.30}$$

where C is the number of classes.

2.4 k-NEAREST NEIGHBORS CLASSIFIER

Let k be a positive integer and $(\mathbf{x}_i, y_i)$, $i = 1, \ldots, n$ be training data. Then, $\mathrm{P}(Y = j \mid \mathbf{X} = \mathbf{x}_0)$, with $\mathbf{x}_0$ as a realization of $\mathbf{X}$, is estimated as follows:

1. Identify the k training data that are closest to $\mathbf{x}_0$. Denote these as $N_k(\mathbf{x}_0)$.

2. Estimate the following quantity:

$$\widehat{\mathrm{P}(Y = j \mid \mathbf{X} = \mathbf{x}_0)} = \frac{1}{k} \sum_{i \in N_k(\mathbf{x}_0)} \mathbb{1}(y_i = j), \quad j = 1, \ldots, C, \tag{2.31}$$

where $\mathbb{1}(\cdot)$ is the indicator function.

3. Classify $\mathbf{x}_0$ to the class with the largest estimated probability (0-1 loss, threshold 0.5 for binary classification).

The choice of k is crucial for the performance of the method. This is connected to the bias-variance trade-off. We identify two cases:

- With *large values of k* we have a simpler decision surface, thus less flexible, but it may generalize well to new data. However, we also have high bias and low variance.
- With *small values of k* the surface is more local, more flexible/jiggly and may not generalize well to new data. The bias-variance relationship is reversed, with low bias and high variance.

2.4.1 Error rate on test data

In order to find the optimal trade-off between bias and variance, the simplest approach is to calculate the error rate on unseen test data, *i.e.*:

1. Calculate probabilities on test data for a range of values of k.
2. Use the response values of the test data to calculate error rate.
3. Choose the value of k that minimizes the error rate.

2.5 Logistic regression

This method is the equivalent of linear regression for classification problems. Why can't we use linear regression for classification problems? Let's discuss some examples to understand why.

Examples.

1. Suppose we want to predict medical condition of patients arriving at emergency based on their symptoms. Assume that the response Y can take only three possible values:

$$Y = \begin{cases} 1 & \text{if the patient has a stroke,} \\ 2 & \text{if the patient has a drug overdose,} \\ 3 & \text{if the patient has an epileptic seizure} \end{cases} \tag{2.32}$$

and that the symptoms are represented by a vector $\mathbf{x}$. If we use linear regression to predict Y based on $\mathbf{x}$, *i.e.*, we try to fit a line to the data, we have two problems:

- We are implying an ordering between the values of the response
- We are implying an equal difference between pairs of response values, *e.g.*, the difference between 1 and 2 is the same as the difference between 2 and 3.

2. Let us now consider only a binary response:

$$Y = \begin{cases} 1 & \text{if the patient has a drug overdose,} \\ 0 & \text{if the patient has a stroke.} \end{cases} \tag{2.33}$$

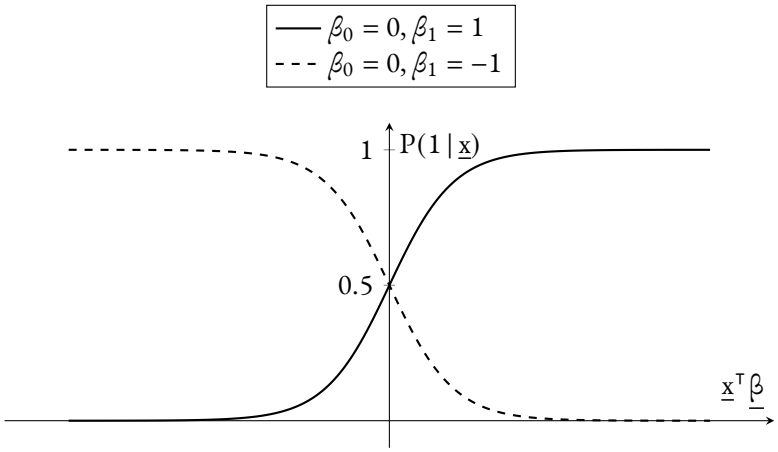

Figure 2.2 *Logistic or* sigmoid *function.*

Since Y is binary, it follows a Bernoulli distribution:

$$f(y) = \begin{cases} p & \text{if } y = 1, \\ 1 - p & \text{if } y = 0. \end{cases} \tag{2.34}$$

Therefore:

$$\mathrm{E}[Y \mid \mathbf{X} = \mathbf{x}] = 1 \cdot \mathrm{P}(1 \mid \mathbf{x}) + 0 \cdot \mathrm{P}(0 \mid \mathbf{x}) = \mathrm{P}(1 \mid \mathbf{x}). \tag{2.35}$$

This means that linear regression makes the assumption:

$$\mathrm{E}[Y \mid \mathbf{X} = \mathbf{x}] = \underbrace{\mathrm{P}(1 \mid \mathbf{x})}_{\in [0,1]} = \beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p = \underbrace{\mathbf{x}^\top \boldsymbol{\beta}}_{\in (-\infty, +\infty)}, \tag{2.36}$$

which is not possible since the left-hand side is a probability and thus must be between 0 and 1.

◇

For logistic regression, we assume:

$$\mathrm{P}(1 \mid \mathbf{x}) = \frac{e^{\mathbf{x}^\top \boldsymbol{\beta}}}{1 + e^{\mathbf{x}^\top \boldsymbol{\beta}}} \in (0, 1). \tag{2.37}$$

This function is called *logistic function* or *sigmoid*.

Example. For $\boldsymbol{\beta} = (\beta_0, \beta_1)$ the explicit function is:

$$\mathrm{P}(1 \mid \mathbf{x}) = \frac{e^{\beta_0 + \beta_1 x_1}}{1 + e^{\beta_0 + \beta_1 x_1}}. \tag{2.38}$$

◇

We can interpret the coefficients in terms of *odds*:

$$\frac{\mathrm{P}(1 \mid \mathbf{x})}{\mathrm{P}(0 \mid \mathbf{x})} = \frac{\mathrm{P}(1 \mid \mathbf{x})}{1 - \mathrm{P}(1 \mid \mathbf{x})} = \frac{\frac{e^{\mathbf{x}^\top \boldsymbol{\beta}}}{1 + e^{\mathbf{x}^\top \boldsymbol{\beta}}}}{1 - \frac{e^{\mathbf{x}^\top \boldsymbol{\beta}}}{1 + e^{\mathbf{x}^\top \boldsymbol{\beta}}}} = e^{\mathbf{x}^\top \boldsymbol{\beta}}. \tag{2.39}$$

If we take the log we get the *log-odds* (or *logit*):

$$\log\left(\frac{\mathrm{P}(1 \mid \mathbf{x})}{1 - \mathrm{P}(1 \mid \mathbf{x})}\right) = \mathbf{x}^\intercal \beta. \tag{2.40}$$

We have now reduced the problem to the estimation of β (parameter estimation). To do so, we rely on the method of *maximum likelihood estimation* (MLE). The likelihood function corresponds to the likelihood of the specific configuration of data we have observed:

$$\mathrm{L}(\beta) = \prod_{i=1}^{n} \mathrm{P}(1 \mid \mathbf{x}_i)^{y_i} (1 - \mathrm{P}(1 \mid \mathbf{x}_i))^{1-y_i}. \tag{2.41}$$

We can also compute the *log-likelihood*:

$$\begin{aligned} \ell(\beta) = \log \mathrm{L}(\beta) &= \sum_{i=1}^{n} y_i \log \mathrm{P}(1 \mid \mathbf{x}_i) + (1 - y_i) \log\bigl(1 - \mathrm{P}(1 \mid \mathbf{x}_i)\bigr) \\ &= \sum_{i=1}^{n} \left(y_i \log\left(\frac{\mathrm{P}(1 \mid \mathbf{x}_i)}{1 - \mathrm{P}(1 \mid \mathbf{x}_i)}\right) + \log\bigl(1 - \mathrm{P}(1 \mid \mathbf{x}_i)\bigr)\right) \\ &= \sum_{i=1}^{n} \left(y_i \mathbf{x}_i^\intercal \beta - \log\bigl(1 + e^{\beta^\intercal \mathbf{x}_i}\bigr)\right). \end{aligned} \tag{2.42}$$

To find the β that maximizes the log-likelihood, *i.e.*, solving:

$$\frac{\partial\, \ell(\beta)}{\partial \beta_j} = 0, \quad j = 0, \ldots, p, \tag{2.43}$$

there is no closed-form solution, unlike for linear regression, for which we have:

$$\hat{\beta} = (X^\intercal X)^{-1} X^\intercal Y, \tag{2.44}$$

so the equations can only be solved numerically (for example, using an adaptation of the Newton-Raphson method).

At the end of the estimation we get different quantities of interest:

1. The estimated coefficients of the model: $\hat{\beta}_0, \hat{\beta}_1, \ldots, \hat{\beta}_p$.
2. The p-values and H_0 hypothesis tests for the significance of each coefficient:

$$H_0 = \begin{cases} \beta_j = 0, \\ \beta_j \neq 0, \end{cases} \qquad \text{for } j = 0, \ldots, p. \tag{2.45}$$

3. The prediction:

$$\widehat{\mathrm{P}(1 \mid \mathbf{x})} = \frac{e^{\hat{\beta}^\intercal \mathbf{x}}}{1 + e^{\hat{\beta}^\intercal \mathbf{x}}}. \tag{2.46}$$

4. Classification, ROC curve on test data, etc.

2.6 Generalized Linear Models

We have already discussed two types of models:

- *Linear regression*, to predict continuous responses. In this case we want to find the distribution of:

$$Y \mid \mathbf{X} = \mathbf{x} \sim \mathcal{N}(\mathbf{x}^\intercal \beta, \sigma^2). \tag{2.47}$$

  In doing this, we are assuming three things:

  1. Normality of the response variable Y.
  2. Linearity over β.
  3. Homoscedasticity, *i.e.*, constant variance σ^2 for all $\mathbf{x}$.

- *Logistic regression*, to predict binary responses. In this case we assume that:

$$Y \mid \mathbf{X} = \mathbf{x} \sim \text{Bernoulli}\underbrace{\left(\frac{e^{\mathbf{x}^\intercal \beta}}{1 + e^{\mathbf{x}^\intercal \beta}}\right)}_{\text{P}(1 \mid \mathbf{x})}. \tag{2.48}$$

Let's denote with Y_i the response variable conditioned on the i-th observation $\mathbf{x}_i$:

$$Y_i \coloneqq Y \mid \mathbf{X} = \mathbf{x}_i. \tag{2.49}$$

Now we compare two types of models, namely the *linear model* and the GLM.

LM	**GLM**
• $Y_i \sim \mathcal{N}(\mu_i, \sigma^2)$ is called *stochastic component*.	• $Y_i \sim f(y_i; \mu_i, \phi)$, where f is a member of the exponential dispersion family.
• $\text{E}[Y_i] = \mu_i = \eta_i$, where $\eta_i = \mathbf{x}_i^\intercal \beta$ is called *deterministic component*.	• $g(\mu_i) = \eta_i = \mathbf{x}_i^\intercal \beta$, where g is called *link function*.

Different choices of f and g give rise to different types of GLMs.

Example. A GLM with f normal and g identity is just a linear model. ◇

Exponential Dispersion Family

Definition 1 (Exponential Dispersion Family). *A density function belongs to the* exponential dispersion family *if it can be written in the form:*

$$f(y_i; \theta_i, \phi) = \exp\left(\frac{y_i \theta_i - b(\theta_i)}{a_i(\phi)} + c(y_i, \phi)\right) \tag{2.50}$$

for some parameters θ_i and ϕ, and some functions $a_i(\cdot)$, $b(\cdot)$ and $c(\cdot, \cdot)$.

In Equation (2.50):

- θ_i is called *canonical parameter*.
- ϕ is called *dispersion parameter*.

- b is called *cumulant generator*.

Let's focus on the binomial distribution. If we write $Y_i \sim \text{Binomial}(n_i, \pi_i)$, we can connect θ_i and μ_i as follows:

$$\mu_i = n_i \pi_i = n_i \frac{e^{\theta_i}}{1 + e^{\theta_i}} \tag{2.51}$$

and also in the variance:

$$\text{Var}(Y_i) = n_i \pi_i (1 - \pi_i) = n_i \frac{\mu_i}{n_i}\left(1 - \frac{\mu_i}{n_i}\right) = \mu_i\left(1 - \frac{\mu_i}{n_i}\right). \tag{2.52}$$

Note. Here the variance is not constant, since μ_i depends on $\mathbf{x}_i$. Apart from the linear model, for any other GLM heteroscedasticity is naturally embedded in the model itself. ○

In general, one can show that $\mu_i = b'(\theta_i)$.

Example. For the binomial case:

$$b'(\theta_i) = (n_i \log(1 + e^{\theta_i}))' = n_i \frac{e^{\theta_i}}{1 + e^{\theta_i}} = \mu_i. \tag{2.53}$$

◇

The variance also can be expressed in terms of b:

$$\text{Var}(Y_i) = b''(\theta_i) a_i(\phi) = V(\mu_i) a_i(\phi), \tag{2.54}$$

where $V(\mu_i)$ is called *variance function*.

Example. With the Gaussian distribution we have that the variance computed with b is constant, as expected for the linear model. ◇

Choice of link functions We said the link function g is a function such that $g(\mu_i) = \eta_i = \mathbf{x}_i^\intercal \beta$. In this way, we connect the mean of the distribution to the linear predictors and parameters.

Let's now consider a Bernoulli distribution:

$$Y_i \sim \text{Bernoulli}(\pi_i). \tag{2.55}$$

Then $\mu_i = \pi_i \in (0, 1)$ and $\eta_i = \mathbf{x}_i^\intercal \beta \in (-\infty, \infty)$. We would like to have a link function that maps $(0, 1)$ into $(-\infty, \infty)$. To achieve this, we search for the inverse first, *i.e.*, we want a function g^{-1} that maps $(-\infty, \infty)$ into $(0, 1)$. There are mainly two popular choices to construct g^{-1} by taking the cumulative distribution function of a random variable:

1. $g^{-1} = \Phi$, where Φ is the CDF of a $\mathcal{N}(0, 1)$ distribution. Therefore, $g = \Phi^{-1}$ (quantile function). If we choose $f = \text{Bernoulli}$ and $g = \Phi^{-1}$, we get the *probit regression*.
2. Consider a logistic distribution, which has the following density function:

$$f(z; \mu, s) = \frac{\exp((z - \mu)/s)}{s(1 + \exp((z - \mu)/s)^2)}, \tag{2.56}$$

with $z \in (-\infty, \infty)$. Setting $\mu = 0$ and $s = 1$, we get:

$$f(z; 0, 1) = \frac{\exp(z)}{1 + \exp(z)}. \tag{2.57}$$

We conclude that choosing f = Bernoulli and $g = f^{-1}(z; 0, 1)$, we obtain the logistic regression. To write explicitly the link function, we have:

$$\mu_i = f(\eta_i; 1, 0) = \frac{e^{\eta_i}}{1 + e^{\eta_i}} \tag{2.58}$$

and this can be inverted to get:

$$\eta_i = g(\mu_i) = f^-1(\mu_i) = \log\left(\frac{\mu_i}{1 - \mu_i}\right), \tag{2.59}$$

which is called *logit link*.

The second option is named *canonical link function* and is the default option in the `glm` function in `R`. In particular, for this function:

$$\eta_i = \log\left(\frac{\mu_i}{1 - \mu_i}\right) = \theta_i, \tag{2.60}$$

so $\eta_i = \theta_i$. In general, for a canonical link function:

$$\eta_i = g(\mu_i) = \theta_i. \tag{2.61}$$

Since $\mu_i = b'(\theta_i)$, we can also write:

$$g = (b')^{-1}. \tag{2.62}$$

There are other options for the Bernoulli distribution, such as the *complementary log-log link*, etc.

Likelihood of GLMs We know that the log-likelihood is of the form:

$$\ell(\beta, \phi) = \sum_{i=1}^{n} \log f(y_i; \theta_i, \phi). \tag{2.63}$$

In the case of a GLM with a canonical link function, we can write:

$$\begin{aligned} \ell(\beta, \phi) &= \sum_{i=1}^{n} \log f(y_i; \theta_i, \phi) \\ &= \sum_{i=1}^{n} \left(\frac{y_i \theta_i - b(\theta_i)}{a_i(\phi)} + c(y_i, \phi) \right). \end{aligned} \tag{2.64}$$

We also remember that θ_i and μ_i are connected as $\mu_i = b'(\theta_i)$, and that μ_i is connected to β as:

$$g(\mu_i) = \eta_i = \mathbf{x}_i^\top \beta. \tag{2.65}$$

The `glm` function in `R` tries to optimize the function with respect to β using:

- *Fisher scoring* (a variant of the Newton-Raphson method).
- *Iteratively Reweighted Least-Squares* algorithm.

At the end, we get β and the maximum likelihood standard errors.

2.6.1 Simulating data from a GLM

We will consider two models:

1. *Poisson regression*, which provide a count response.
2. *Logistic regression*, which is used for binary responses.

Poisson regression First, let's describe what parameters we have and what we expect. For a variable Y that follows a Poisson distribution, we have:

- $X_1, \dots, X_p$: the predictors.
- $Y_i := Y \mid \mathbf{X} = \mathbf{x}_i,\ i = 1, \dots, n$: the response variable conditioned on the predictors.

$$\begin{array}{c} \\ 1 \\ \vdots \\ n \end{array} \begin{array}{c} \begin{array}{ccc} 1 & \cdots & p \end{array} \\ \begin{bmatrix} x_{11} & \cdots & x_{1p} \\ \vdots & \ddots & \vdots \\ x_{n1} & \cdots & x_{np} \end{bmatrix} \\ X \end{array} \qquad \begin{array}{c} \\ \begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix} \\ Y \end{array} \tag{2.66}$$

Here, $y_1, \dots y_n$ are n observations of our response and are independent but not identically distributed, since the mean of Y_i depends on the index i. We assume $Y_i \sim \text{Poisson}(\mu_i)$. Then we set:

$$g(\mu_i) = \log(\mu_i) = \eta_i = \mathbf{x}_i^\intercal \beta. \tag{2.67}$$

We can rewrite the definition of Y_i as $\text{Poisson}(e^{\mathbf{x}_i^\intercal \beta})$.

Now we want to simulate data from this model. First, we set $p = 1$ and $n = 100$. Then:

1. Generate $x_1, \dots, x_{100}$ from any distribution.
2. We set $\eta_i = \beta_0 + \beta_1 x_i$, $i = 1, \dots, 100$. For example, we can fix $\beta_0 = 1$ and $\beta_1 = 2$.
3. We sample $y_i \sim \text{Poisson}(e^{\eta_i})$, $i = 1, \dots, 100$.

Residuals In linear models we can express the response observation in two different but equivalent ways:

- $y_i = \mathbf{x}_i^\intercal \beta + \varepsilon_i$, where $\varepsilon_i \sim \mathcal{N}(0, \sigma^2)$.
- $y_i \sim \mathcal{N}(\mathbf{x}_i^\intercal \beta, \sigma^2)$.

A natural definition of residuals in linear models is:

$$e_i = y_i - \mathbf{x}_i^\intercal \beta, \quad i = 1, \dots, n. \tag{2.68}$$

For a GLM, we can define the residuals as *pseudo-residuals*:

$$r_i = y_i - \hat{\mu}_i, \quad i = 1, \dots, n, \tag{2.69}$$

A common way to standardize the residuals is to calculate the *Pearson residuals*:

$$r_i^P = \frac{y_i - \hat{\mu}_i}{\sqrt{\frac{\widehat{\text{Var}(y_i)}}{\hat{\phi}}}}, \quad i = 1, \dots, n. \tag{2.70}$$

For the Poisson regression pseudo-residuals are actually:

$$r_i = y_i - \hat{\mu}_i = y_i - e^{\mathbf{x}_i^\top \hat{\beta}}, \tag{2.71}$$

and the Pearson residuals are:

$$r_i^P = \frac{y_i - e^{\mathbf{x}_i^\top \hat{\beta}}}{\sqrt{e^{\mathbf{x}_i^\top \hat{\beta}}}}, \quad i = 1, \dots, n. \tag{2.72}$$

Asymptotically, the Pearson residuals are distributed as a standard normal distribution, but they could be skewed if $\hat{\mu}_i$ is small.

Deviance The deviance of an observation is defined as:

$$d_i = 2(\ell_i(\underbrace{y_i}_{\hat{\mu}_i = y_i}, y_i) - \ell_i(\underbrace{\hat{\mu}_i}_{e^{\mathbf{x}_i^\top \hat{\beta}}}, y_i)) \geqslant 0, \quad i = 1, \dots, n. \tag{2.73}$$

Here we are basically computing $y_i - \hat{\mu}_i$ at the level of the log-likelihood. The general deviance is computed as a sum:

$$D = \sum_{i=1}^{n} d_i \tag{2.74}$$

and:

$$r_i^D = \sqrt{d_i} \cdot \operatorname{sgn}(y_i - \hat{\mu}_i) = \begin{cases} \sqrt{d_i} & \text{if } y_i \geqslant \hat{\mu}_i, \\ -\sqrt{d_i} & \text{if } y_i < \hat{\mu}_i. \end{cases} \tag{2.75}$$

2.7 Discriminant analysis methods

Let's analyze the following situation, in which there are Y categorical variable and K classes. We want to find an alternative to logistic regression and k-NN. Similar to the former, *discriminant analysis methods* are parametric methods and are among parametric classification models. The aim is to estimate the probability:

$$\mathrm{P}(Y = j \mid \mathbf{X} = \mathbf{x}), \quad j = 1, \dots, K. \tag{2.76}$$

For discriminant analysis methods, these probabilities are estimated "indirectly" via:

$$f_j(\mathbf{x}) = \begin{cases} \mathrm{P}(\mathbf{X} = \mathbf{x} \mid Y = j) & \text{if } \mathbf{X} \text{ is discrete,} \\ f(\mathbf{x} \mid Y = j) & \text{if } \mathbf{X} \text{ is continuous.} \end{cases} \tag{2.77}$$

We call $f_j(\mathbf{x})$ the *joint distribution* of $\mathbf{X}$ given $Y = j$, $j = 1, \dots, K$. Via Bayes' theorem:

$$\mathrm{P}(Y = j \mid \mathbf{X} = \mathbf{x}) = \frac{\mathrm{P}(\mathbf{X} = \mathbf{x} \mid Y = j)\,\mathrm{P}(Y = j)}{\sum_{k=1}^{K} \mathrm{P}(\mathbf{X} = \mathbf{x} \mid Y = k)\,\mathrm{P}(Y = k)} = \frac{f_j(\mathbf{x})\pi_j}{\sum_{k=1}^{K} f_k(\mathbf{x})\pi_k} \tag{2.78}$$

where $\pi_j = \mathrm{P}(Y = j)$ is the *prior probability* of class j. We assign $\mathbf{x}$ to the class j that maximizes:

$$\mathrm{P}(Y = j \mid \mathbf{X} = \mathbf{x}), \quad j = 1, \dots, K, \tag{2.79}$$

that is "assign $\mathbf{x}$ to class j that maximizes":

$$f_j(\mathbf{x})\pi_j, \quad j = 1, \dots, K. \tag{2.80}$$

Estimation of π_j There are two ways of estimating π_j from data:

1. $\hat{\pi}_j = \frac{1}{K}, \ j = 1, \ldots, K.$ (Classes equally likely)
2. $\hat{\pi}_j = \frac{n_j}{n}, \ j = 1, \ldots, K.$ (Fractional counts)

The various discriminant analysis methods distinguish themselves in the assumptions that they make on $f_j(\mathbf{x})$. We can deal with:

1. Linear Discriminant Analysis (LDA).
2. Quadratic Discriminant Analysis (QDA).
3. Naive Bayes.

2.7.1 LDA with $p = 1$

Linear Discriminant Analysis assumes:

1. $f_j(x)$ is Gaussian (conditional on $Y = j$):

$$f_j(x) = \frac{1}{\sqrt{2\pi}\sigma_j} e^{-\frac{1}{2}\left(\frac{x-\mu_j}{\sigma_j}\right)^2}, \quad j = 1, \ldots, K. \tag{2.81}$$

2. $\sigma_1^2 = \sigma_2^2 = \cdots = \sigma_K^2 = \sigma^2.$ (Homoscedasticity)

To summarize those assumptions, we can say:

$$X \mid Y = j \sim \mathcal{N}(\mu_j, \sigma^2), \quad j = 1, \ldots, K. \tag{2.82}$$

What about linearity? Plugging $f_j(x)$ in the Bayes formula, we obtain:

$$P(Y = j \mid X = x) = \frac{\frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{1}{2}\left(\frac{x-\mu_j}{\sigma}\right)^2} \pi_j}{\sum_{k=1}^{K} \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{1}{2}\left(\frac{x-\mu_k}{\sigma}\right)^2} \pi_k}, \quad j = 1, \ldots, K. \tag{2.83}$$

Assigning x to class j that maximizes $\mathrm{P}(Y = j \mid X = x)$ is equivalent to assigning x to class j that maximizes:

$$e^{-\frac{1}{2}\left(\frac{x-\mu_j}{\sigma}\right)^2} \pi_j, \quad j = 1, \ldots, K, \tag{2.84}$$

which, in turn, is equivalent to maximizing:

$$\begin{aligned} \hat{\delta}_j(x) &= \log\left(e^{-\frac{1}{2}\left(\frac{x-\mu_j}{\sigma}\right)^2} \pi_j\right) \\ &= -\frac{1}{2}\left(\frac{x-\mu_j}{\sigma}\right)^2 + \log(\pi_j) \\ &\propto -\frac{1}{2}\frac{\mu_j^2}{\sigma^2} + \log(\pi_j) + x\frac{\mu_j}{\sigma^2}. \end{aligned} \tag{2.85}$$

We see that this expression is linear in x.

Estimation of parameters Parameters are estimated via maximum likelihood:

$$\begin{aligned}\hat{\mu}_j &= \frac{\sum_{i \text{ s.t. } y_i = j} x_i}{n_j}, \quad j = 1, \dots, K, \quad \text{(Sample mean of } X \text{ in class } j\text{)} \\ \hat{\sigma}^2 &= \frac{\sum_{i=1}^K (n_i - 1) s_i^2}{\sum_{i=1}^K (n_i - 1)} = \frac{\sum_{i=1}^K \sum_{j \text{ s.t. } y_j = i} (x_j - \hat{\mu}_i)^2}{n - K}. \quad \text{(Pooled variance)}\end{aligned} \tag{2.86}$$

Then assign x to class j that maximizes:

$$\hat{\delta}_j(x) = \log(\hat{\pi}_j) - \frac{1}{2}\frac{\hat{\mu}_j^2}{\hat{\sigma}^2} + x\frac{\hat{\mu}_j}{\hat{\sigma}^2}, \quad j = 1, \dots, K. \tag{2.87}$$

2.7.2 LDA with $p \geqslant 1$

Here we consider a multivariate extension of LDA. Therefore, the variables are now $\mathbf{X} = (x_1, \dots, x_p)$. In this case, LDA assumes:

1. $\mathbf{X}$ has a multivariate Gaussian distribution conditional on $Y = j$:

$$f_j(\mathbf{x}) = \frac{1}{(2\pi)^{p/2}\left|\Sigma_j\right|^{1/2}} \exp\left(-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu}_j)^\intercal \Sigma_j^{-1}(\mathbf{x} - \boldsymbol{\mu}_j)\right), \quad j = 1, \dots, K, \tag{2.88}$$

   with $\boldsymbol{\mu}_j = (\mu_{j1}, \dots, \mu_{jp})^\intercal$ vector of means and Σ_j $p \times p$ covariance matrix.

2. $\Sigma_1 = \Sigma_2 = \dots = \Sigma_K = \Sigma = \begin{pmatrix} \sigma_1^2 & \sigma_{12} & \cdots & \sigma_{1p} \\ \sigma_{21} & \sigma_2^2 & \cdots & \sigma_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ \sigma_{p1} & \sigma_{p2} & \cdots & \sigma_p^2 \end{pmatrix}$, where $\sigma_{jl} = \operatorname{Cov}(X_j, X_l \mid Y = k)$ and $\sigma_j^2 = \operatorname{Var}(X_j \mid Y = k)$.

Linear decision surface Let's analyze the decision surface as in the univariate case.

$$P(Y = j \mid \mathbf{X} = \mathbf{x}) = \frac{\frac{1}{(2\pi)^{p/2}|\Sigma|^{1/2}} \exp\left(-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu}_j)^\intercal \Sigma^{-1}(\mathbf{x} - \boldsymbol{\mu}_j)\right)\pi_j}{\sum_{k=1}^K \frac{1}{(2\pi)^{p/2}|\Sigma|^{1/2}} \exp\left(-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu}_k)^\intercal \Sigma^{-1}(\mathbf{x} - \boldsymbol{\mu}_k)\right)\pi_k} \tag{2.89}$$

So assigning $\mathbf{x}$ to class j that maximizes the numerator is equivalent to assigning $\mathbf{x}$ to class j that maximizes:

$$\begin{aligned}\delta_j(\mathbf{x}) &= -\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu}_j)^\intercal \Sigma^{-1}(\mathbf{x} - \boldsymbol{\mu}_j) + \log(\pi_j) \\ &= \underbrace{-\frac{1}{2}\mathbf{x}^\intercal \Sigma^{-1}\mathbf{x}}_{\text{Constant w.r.t. } j} - \frac{1}{2}\boldsymbol{\mu}_j^\intercal \Sigma^{-1}\boldsymbol{\mu}_j + \mathbf{x}^\intercal \Sigma^{-1}\boldsymbol{\mu}_j + \log(\pi_j) \\ &\propto \underbrace{-\frac{1}{2}\boldsymbol{\mu}_j^\intercal \Sigma^{-1}\boldsymbol{\mu}_j + \log(\pi_j)}_{(\beta_0)_j} + \mathbf{x}^\intercal \underbrace{\Sigma^{-1}\boldsymbol{\mu}_j}_{(\beta)_j},\end{aligned} \tag{2.90}$$

which is indeed linear. So the $\mathbf{x}$s such that $\delta_1(\mathbf{x}) = \delta_2(\mathbf{x})$ correspond to a *linear decision surface*.

2.7.3 Quadratic discriminant analysis

Quadratic Discriminant Analysis relaxes the second assumption. Therefore, the assumption is now only:

$$\mathbf{X} \mid Y = j \sim \mathcal{N}_p(\boldsymbol{\mu}_j, \Sigma_j), \quad j = 1, \dots, K. \tag{2.91}$$

The model is more complex, so it is more flexible, but at the expense of many more parameters (relates to the bias-variance tradeoff).

Quadratic decision surface Assign $\mathbf{x}$ to class j that maximizes:

$$\begin{aligned} \delta_j(\mathbf{x}) &= -\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu}_j)^\top \Sigma_j^{-1}(\mathbf{x} - \boldsymbol{\mu}_j) - \frac{1}{2}\log|\Sigma_j| + \log(\pi_j) \\ &= -\frac{1}{2}\mathbf{x}^\top \Sigma_j^{-1}\mathbf{x} - \frac{1}{2}\boldsymbol{\mu}_j^\top \Sigma_j^{-1}\boldsymbol{\mu}_j + \mathbf{x}^\top \Sigma_j^{-1}\boldsymbol{\mu}_j - \frac{1}{2}\log|\Sigma_j| + \log(\pi_j) \\ &= \underbrace{\log(\hat{\pi}_j) - \frac{1}{2}\log|\hat{\Sigma}_j| - \frac{1}{2}\hat{\boldsymbol{\mu}}_j^\top \hat{\Sigma}_j^{-1}\hat{\boldsymbol{\mu}}_j}_{\text{Intercept}} + \underbrace{\mathbf{x}^\top \hat{\Sigma}_j^{-1}\hat{\boldsymbol{\mu}}_j}_{\text{Linear}} - \underbrace{\frac{1}{2}\mathbf{x}^\top \hat{\Sigma}_j^{-1}\mathbf{x}}_{\text{Quadratic}}. \end{aligned} \tag{2.92}$$

This leads to a quadratic decision surface.

Estimation of parameters Recall:

$$\begin{aligned} \hat{\boldsymbol{\mu}}_j &= \frac{1}{n_j} \sum_{i \text{ s.t. } y_i = j} \mathbf{x}_i, \quad j = 1, \dots, K, \quad \text{(Vector of sample means)} \\ \underbrace{\hat{\Sigma}_j}_{p \times p} &= \frac{1}{n_j - 1} \sum_{i \text{ s.t. } y_i = j} (\mathbf{x}_i - \hat{\boldsymbol{\mu}}_j)(\mathbf{x}_i - \hat{\boldsymbol{\mu}}_j)^\top, \quad j = 1, \dots, K. \quad \text{(Sample covariance)} \end{aligned} \tag{2.93}$$

This leads to a quadratic decision surface:

$$\mathbf{x} \colon \hat{\delta}_1(\mathbf{x}) = \hat{\delta}_2(\mathbf{x}) \tag{2.94}$$

and estimated probabilities:

$$\widehat{\mathrm{P}(Y = 1 \mid \mathbf{X} = \mathbf{x})} = \frac{e^{\hat{\delta}_1(\mathbf{x})}}{e^{\hat{\delta}_1(\mathbf{x})} + e^{\hat{\delta}_2(\mathbf{x})}}. \quad \text{(For two classes)} \tag{2.95}$$

LDA vs QDA QDA has many more parameters:

- LDA has $Kp + p(p+1)/2$ parameters: $\boldsymbol{\mu}_1, \dots, \boldsymbol{\mu}_K$ and Σ.
- QDA has $Kp + Kp(p+1)/2$ parameters: $\boldsymbol{\mu}_1, \dots, \boldsymbol{\mu}_K$ and $\Sigma_1, \dots, \Sigma_K$.

Example. For example, with $K = 2$, $p = 50$, LDA has 1375 parameters, while QDA has 2650 parameters. Any choice between the two models needs to involve that data. ◇

LDA vs LR Let's compare Linear Discriminant Analysis and Logistic Regression.

- Both lead to a linear decision surface.
- LDA can be naturally applied for any number of classes.
- LDA is more restrictive as it assumes that $\mathbf{X}$ is a multivariate Gaussian conditional on the class, so this needs to be checked.
- LDA is computationally more stable when probabilities are closer to 0 or 1.

2.7.4 Naive Bayes classifier

In this setting we are dealing with a categorical variable Y and K classes. Naive Bayes belongs to the family of discriminant analysis methods. We recall how to estimate $\mathrm{P}(Y = j \mid \mathbf{X} = \mathbf{x})$:

$$\mathrm{P}(Y = j \mid \mathbf{X} = \mathbf{x}) = \frac{\pi_j f_j(\mathbf{x})}{\sum_{i=1}^{K} \pi_i f_i(\mathbf{x})}, j = 1, \ldots, K. \tag{2.96}$$

We also recall that discriminant analysis methods make assumptions on $f_j(\mathbf{x})$:

1. LDA, $f_i(\mathbf{x}) = \mathcal{N}_p(\boldsymbol{\mu}_j, \Sigma)$. Normality and homoscedasticity give rise to linear decision surfaces.
2. QDA, $f_i(\mathbf{x}) = \mathcal{N}_p(\boldsymbol{\mu}_j, \Sigma_j)$. Normality and heteroscedasticity give rise to quadratic decision surfaces.

A third possible model is the Naive Bayes classifier. The main assumption on $f_i(\mathbf{x})$ is:

$$f_j(\mathbf{x}) = \prod_{l=1}^{p} f_{jl}(x_l), \tag{2.97}$$

with $f_{jl}(x_l)$ the distribution of X_l conditional on $Y = j$. In other words, predictors $x_1, \ldots, x_p$ are independent conditional on $Y = j$. If $\mathbf{X}$ are discrete variables, then:

$$\mathrm{P}(X_1 = x_1, \ldots, X_p = x_p \mid Y = j) = \underbrace{\mathrm{P}(X_1 = x_1 \mid Y = j)}_{f_{j1}(x_1)} \cdots \underbrace{\mathrm{P}(X_p = x_p \mid Y = j)}_{f_{jp}(x_p)}. \tag{2.98}$$

Gaussian Naive Bayes We assume:

1. $\mathbf{X} \mid Y = j \sim \mathcal{N}_p(\boldsymbol{\mu}_j, \Sigma_j)$. (Gaussianity)
2. $X_i \perp X_l \mid Y = j \quad \text{for } i \neq l.$ (Conditional independence)

Proposition. *Under a Gaussian assumption:*

$$W \perp Z \iff \mathrm{Cov}(W, Z) = 0. \tag{2.99}$$

Note. The ($\Rightarrow$) direction is true for any distribution, while the ($\Leftarrow$) direction is true only for Gaussian distributions. ○

Therefore, 1 and 2 together mean that:

$$\Sigma_j = \begin{pmatrix} \sigma_{j1}^2 & 0 & \cdots & 0 \\ 0 & \sigma_{j2}^2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \sigma_{jp}^2 \end{pmatrix}. \tag{2.100}$$

The covariance matrix is diagonal and depends on j, with $\sigma_{jl}^2 = \mathrm{Var}(X_l \mid Y = j)$.

Compared to Naive Bayes:

1. LDA has variances that do not depend on j (*i.e.*, values along the diagonal are the same across classes), but non-zero values off-diagonal.
2. QDA has non-zero class-specific off-diagonal values.

Naive Bayes decision surface We know from the previous section that:

$$\begin{aligned} \mathrm{P}(Y = j \mid \mathbf{X} = \mathbf{x}) &\propto \pi_j f_j(\mathbf{x}) \\ &= \pi_j \prod_{l=1}^{p} f_{jl}(x_l) \\ &= \pi_j \prod_{l=1}^{p} \frac{1}{\sqrt{2\pi\sigma_{jl}^2}} e^{-\frac{1}{2}\left(\frac{x_l - \mu_{jl}}{\sigma_{jl}}\right)^2}. \end{aligned} \tag{2.101}$$

Therefore, the decision surface is built from:

$$\delta_j(\mathbf{x}) = \log(\pi_j f_j(\mathbf{x})) = \log(\pi_j) - \frac{1}{2}\sum_{l=1}^{p} \log(\sigma_{lj}^2) - \frac{1}{2}\sum_{l=1}^{p}\left(\frac{x_l - \mu_{jl}}{\sigma_{jl}}\right)^2. \tag{2.102}$$

We can see that we have a simpler quadratic form compared to QDA ($\mathbf{x}^\intercal \Sigma_j^{-1}\mathbf{x}$), since it does not require the inversion of Σ_j.

Multinomial Naive Bayes classifier Consider $X_1, \ldots, X_p$ categorical variables. We also consider $f_j(\mathbf{x}) = \mathrm{P}(\mathbf{X} = \mathbf{x} \mid Y = j)$ in this case. How many parameters are involved?

Example. Analyze the situation with two classes, $j = 1, 2$ and p predictors $\{x_i\}$, each of which has G_i categories, with $i = 1, \ldots, p$. We have:

$$\mathrm{P}(Y = j \mid \mathbf{X} = \mathbf{x}) \propto \mathrm{P}(Y = j)\,\mathrm{P}(X_1 = x_1 \mid Y = j) \cdots \mathrm{P}(X_p = x_p \mid Y = j). \tag{2.103}$$

For each term $\mathrm{P}(X_i = x_k \mid Y = j)$, we need to estimate parameters in the form θ_{ijk}. For example, for $i = 1$:

$$\begin{aligned} \theta_{111} &= \mathrm{P}(X_1 = 1 \mid Y = 1) \\ &\vdots \\ \theta_{11G_1} &= \mathrm{P}(X_1 = G_1 \mid Y = 1) \\ \theta_{121} &= \mathrm{P}(X_1 = 1 \mid Y = 2) \\ &\vdots \\ \theta_{12G_1} &= \mathrm{P}(X_1 = G_1 \mid Y = 2). \end{aligned} \tag{2.104}$$

We would need to estimate parameters also for $i = 2, \ldots, p$ in this example. ◇

If we assume global independence of parameters between predictors, the estimation of vectors of probability can be done separately. Moreover, we estimate the single θ_{ijk} under local independence of parameters. Therefore, we can estimate the distribution of $X_i \mid Y = j$ from data separately for each predictor X_i and class j.

2.8 Bayesian inference

We set $Z \coloneqq X_i \mid Y = j$ and we call $f(z; \theta)$ distribution of Z. We have access to some data $z_1, \ldots, z_n$ and we want to estimate θ from that data. The most common approach is by maximizing the likelihood:

$$\mathrm{L}(\theta) = \prod_{i=1}^{n} f(z_i; \theta). \tag{2.105}$$

Then:

$$\hat{\theta} = \operatorname*{argmax}_{\theta} \mathrm{L}(\theta) \tag{2.106}$$

as the MLE of θ. Doing this, we use θ as unknown but fixed constants.

Bayesian Philosophy In Bayesian inference, θ is in fact a random variable which has a *prior* distribution $h(\theta)$, which represents the knowledge about θ we have before we see the data. Given the data, we update this distribution:

$$h(\theta) \longrightarrow g(\theta \mid \text{data}), \tag{2.107}$$

which is called the *posterior* distribution and combines prior knowledge and information from data. In practice:

$$g(\theta \mid \text{data}) = \frac{f(\text{data} \mid \theta) h(\theta)}{\mathrm{P}(\text{data})} \propto f(\text{data} \mid \theta) h(\theta). \tag{2.108}$$

We can see the posterior distribution as a refinement of the prior distribution, which is updated with the information from data (likelihood).

There are simple cases where the shape of $g(\theta \mid \text{data})$ is known, so Bayesian inference estimates parameters of the posterior distribution. However, in most cases we have no knowledge about g, and Markov Chain Monte Carlo (MCMC) sampling is needed to draw samples from g.

Example. Beta-binomial. Consider:

- Z binary (remembering that $Z \coloneqq X_i \mid Y = j$).
- $Z \sim \text{Bernoulli}(\theta)$, with $0 \leqslant \theta \leqslant 1$ and $\theta = \mathrm{P}(Z = 1)$.
- $z_1, \ldots, z_n$ i.i.d. samples from Z.

Therefore:

$$\begin{aligned} \mathrm{L}(\theta) &= \prod_{i=1}^{n} \theta^{z_i} (1-\theta)^{1-z_i} \\ &= \theta^{\sum_{i=1}^{n} z_i} (1-\theta)^{n - \sum_{i=1}^{n} z_i}. \end{aligned} \tag{2.109}$$

In the classical frequentist approach:

$$\hat{\theta}^{(\mathrm{MLE})} = \operatorname*{argmax}_{\theta} \mathrm{L}(\theta) = \frac{1}{n} \sum_{i=1}^{n} z_i. \quad \text{(Empirical frequencies)} \tag{2.110}$$

In Bayesian inference, we need to set a prior distribution:

$$h(\theta) = \frac{\Gamma(\alpha + \beta)}{\Gamma(\alpha)\Gamma(\beta)} \theta^{\alpha - 1} (1-\theta)^{\beta - 1}, \tag{2.111}$$

with α and β fixed according to prior knowledge. Then:

$$\begin{aligned} g(\theta \mid \text{data}) &\propto \mathrm{L}(\theta) h(\theta) \\ &= \theta^{\sum_{i=1}^{n} z_i} (1-\theta)^{n - \sum_{i=1}^{n} z_i} \cdot \theta^{\alpha - 1} (1-\theta)^{\beta - 1} \\ &= \theta^{\sum_{i=1}^{n} z_i + \alpha - 1} (1-\theta)^{n - \sum_{i=1}^{n} z_i + \beta - 1}. \end{aligned} \tag{2.112}$$

We recognize $g(\theta \mid \text{data})$ as a Beta distribution with:

$$\begin{aligned} \alpha^* &= \alpha + \sum_{i=1}^{n} z_i \\ \beta^* &= \beta + n - \sum_{i=1}^{n} z_i \end{aligned} \tag{2.113}$$

Technically, we say that the Beta prior is *conjugate* to the binomial likelihood. ◇

Choice of prior The choice of the prior distribution is a crucial step in Bayesian inference. Since the expected value of a Beta distribution is $\alpha/(\alpha+\beta)$, we can use this information to set those parameters. It is quite common to set $\alpha = \beta = 1$, which corresponds to a uniform distribution on $[0, 1]$ and, therefore, to a non-informative prior. In this case:

$$g(\theta \mid \text{data}) = \text{Beta}\left(\sum_{i=1}^{n} z_i + 1, n - \sum_{i=1}^{n} z_i + 1\right). \tag{2.114}$$

So:

$$\hat{\theta}^{(\text{Bayesian})} = \frac{\sum_{i=1}^{n} z_i + 1}{n + 2}. \tag{2.115}$$

The increments at the numerator and denominator are also called *imaginary counts*. They are useful to avoid zero probabilities or divisions by zero.

For a more general categorical variable, the parameter θ becomes a vector of probabilities $\boldsymbol{\theta}$, the binomial distribution becomes a *multinomial distribution* and the Beta distribution becomes a *Dirichlet distribution*.

3 Optimizing models

3.1 Resampling methods

We have seen a number of classifiers, such as LR, LDA, QDA, Naive Bayes, k-NN. In this chapter, we will see how to choose the optimal model among a set of candidate models. We will also see how to estimate the test error rate of a model.

Different models are associated to a different level of complexity (*e.g.*, number of parameters, k, etc.) and, therefore, a different trade-off between bias and variance. We recall that we can have different degrees of complexity of decision surfaces in the classification setting:

- *Simple decision surfaces* are given by k-NN with large k, LDA, parametric models with small number of parameters. These models have high bias and low variance.
- *Complex decision surfaces* are given by k-NN with small k and parametric models with many parameters. These models have low bias and high variance.

This reasoning can be done also in the regression setting. In polynomial regression, we can vary the degree of the polynomial to obtain different levels of complexity of the decision surface. For instance, we can set d equals to the number of observations minus 1 to fit the data perfectly, but this will lead to overfitting.

Measure accuracy on unseen data The best solution is to use a large, designated test set, left aside to select the optimal model. However, often we cannot have access to a separated dataset that can be used as test set only. To overcome this problem we can rely on two different approaches:

1. Make some adjustments to the training error in order to estimate the error rate. This is done through Akaike Information Criterion (AIC), Bayesian Information Criterion (BIC), etc. Those methods can be used only for parametric models, as the likelihood is needed to compute the criteria, and they are called *asymptotic*.
2. *Resampling methods*: estimate directly the test error rate by devising some resampling strategy.

Resampling methods The general idea is to hold out a subset of the data from the fitting process, so that we can use these observations to test the fitted model. There are various (simpler and more complex) versions of this idea, that try to reach a compromise between:

1. Computational complexity.
2. Using enough data for fitting and testing.
3. Accounting for variations due to specific sampling.

Different strategies

1. *Validation set approach.* We randomly divide the dataset into two parts: training set and validation set. Typical split proportions are 50-50 and 70-30. Then, the model is fitted on training data and tested on validation data.

   Examples.

   - *Polynomial regression*
     (a) Fix d.
     (b) Get $\hat{\beta}$ on training data.
     (c) Compute $\hat{y}_i = \mathbf{x}_i^\intercal \hat{\beta}$, $i = 1, \ldots, n$ on validation data ($\mathbf{x}_i \in$ Validation set).
     (d) Compare $\hat{y}_i$ with the true y_i (calculate MSE on validation data).
     (e) Do this across a grid of values for d and choose d that achieves the minimum MSE.
   - *k-NN*
     (a) Fix k.
     (b) Calculate $\mathrm{P}(1 \mid \mathbf{x}_i)$ for $\mathbf{x}_i \in$ Validation set using k-NN on training data.
     (c) Calculate error rate by comparing $\hat{y}_i$ with y_i.
     (d) Choose optimal k on a grid of values. ◇

   Some problems with this approach are:

   - Variability for different data splits (but we can repeat the procedure multiple times and average the results).
   - Not using all the data to fit and all the data to test the model.

2. *k-fold cross-validation.* With this approach we randomly divide data into k equally sized parts (typical values are $k = 5$ or $k = 10$). Then:
   (a) Leave out the k-th fold.
   (b) Fit the model on the remaining $k - 1$ folds.
   (c) Use the model to make predictions on the left-out fold.
   (d) Repeat across k folds and combine the results.

   Formally:
   (a) Fix a model (*i.e.*, set parameters k, d, *etc.*).
   (b) Let $C_1, \ldots, C_k$ be the indices of the observations in each fold.
   (c) For each fold C_i, calculate the error:

$$E_i = \begin{cases} \mathrm{MSE}_i : \dfrac{1}{|C_i|} \displaystyle\sum_{j \in C_i} (y_j - \hat{y}_j)^2 & \text{if regression,} \\ \mathrm{ER}_i : \dfrac{1}{|C_i|} \displaystyle\sum_{j \in C_i} \mathbb{1}(\hat{y}_j \neq y_j) & \text{if classification,} \end{cases} \qquad i = 1, \ldots, k. \tag{3.1}$$

(d) Then combine the results across folds:

$$\text{CV} = \frac{1}{k}\sum_{i=1}^{k} E_i. \tag{3.2}$$

With k-fold CV we can benefit from the following advantages:

- Both fitting and testing see the entire dataset.
- Low variability of CV error.

However, performing this is computationally more expensive than a single train-validation split.

Choice of k Some key points to consider when choosing k are:

- Empirically, $k = 5$ or $k = 10$ are good choices.
- $k = n$ is the largest admissible value and each fold is made of a single observation. This variant is called *Leave-One-Out Cross Validation* (LOOCV). It is computationally expensive in general, except for linear regression where there is a closed-form formula to compute the LOOCV error without fitting n different models:

$$\text{LOOCV} = \frac{1}{n}\sum_{i=1}^{n}\left(\frac{y_i - \hat{y}_i}{1 - h_i}\right)^2, \tag{3.3}$$

where $\hat{y}_i$ is the prediction made using $\hat{\boldsymbol{\beta}}$ fitted on full data and h_i is the i-th diagonal element of the *hat matrix* $H = X(X^\top X)^{-1}X^\top$, *i.e.*, $h_i = H_{ii}$.

Note. The h_i term is also called *leverage* and it measures the influence of the i-th observation on its own predicted value. The H matrix is referred to as *hat matrix* because it maps the observed $\mathbf{y}$ to the fitted $\hat{\mathbf{y}}$, *i.e.*, $\hat{\mathbf{y}} = H\mathbf{y}$. ○

A problem related to LOOCV is that it has higher variability compared to lower-valued k-fold CV, because the training sets are very similar to each other (differing by only one observation produces highly correlated subsets).

Hyperparameter tuning vs. Model comparison Cross-validation can be used both for hyperparameter tuning (*i.e.*, the choice of d, k, …) and for model comparison.

Example. Let's compare k-NN and logistic regression in a setting with $X_1, \ldots, X_p$ predictors. If we perform CV on full data and vary the parameter k we obtain two curves, one for k-NN and one for logistic regression (the latter will be constant as it doesn't depend on k). However, this comparison could be a bit unfair towards logistic regression since the CV procedure to select k has seen the full data. ◇

Nested cross-validation To overcome the problem of unfair comparison, we can use *nested cross-validation*:

1. Take a fold out.
2. Perform k-fold CV on the remaining $k - 1$ folds to select the tuning parameters.
3. Use the optimal model to predict fold left out in step 1 and calculate the error.
4. Iterate across all folds.

3.2 Model selection for linear models

As we saw, model selection can be performed by evaluating the fitted model on unseen data (*e.g.*, with CV). Alternatively, for parametric models, for which we can define a likelihood function, there are criteria that make adjustments to training error (likelihood) by accounting for model complexity. These criteria are:

1. Akaike Information Criterion (AIC):

$$\mathrm{AIC}(M) = -2\log \mathrm{L} + 2d, \tag{3.4}$$

where $\log \mathrm{L}$ is the log-likelihood of the model M and d is the number of parameters in the model (which tells its complexity).

Observation. If we only had the log-likelihood term, we would always select the most complex model. In fact, minimizing the error is equivalent to maximizing the likelihood, and the more complex the model, the better it fits the data. △

Considering different models M_i, $i = 1, \dots, m$, we choose the model with the lowest AIC value.

2. Bayesian Information Criterion (BIC):

$$\mathrm{BIC}(M) = -2\log \mathrm{L} + \log(n)d, \tag{3.5}$$

where n is the number of observations. Typically, $\log(n) > 2$ for $n > 7$. This criterion puts more weight on model complexity, so in the end we tend to select simpler models than with AIC.

Some remarks:

1. The true model may note be in the family of models we consider (called *misclassification*).
2. Model selection depends on which and how much data we have. For example, the true generating process of a dataset may be a polynomial of high degree, but we may not have enough data to fit this model robustly. This problem is even more pronounced in high-dimensional cases ($p \gg n$) when even a simple linear model may not be possible to fit.
3. The bias-variance trade-off is connected also to the bias and variance of β.

Linear models Consider a linear model consisting of:

- Y response.
- $X_1, \dots, X_p$ predictors.
- ε error term, with $\varepsilon \sim \mathcal{N}(0, \sigma^2)$.

Formally we can write:

$$Y = \beta_1 \mathbf{x}_1 + \cdots + \beta_p \mathbf{x}_p + \varepsilon = \mathbf{x}^\intercal \beta + \varepsilon, \tag{3.6}$$

We also have a dataset $(\mathbf{x}_i, y_i)$, $i = 1, \dots, n$. Our aim is to estimate $\beta = (\beta_1 \cdots \beta_p)^\intercal$ from data. We proceed with a least squares approach. In matrix form we can write:

$$\begin{gathered} \mathbf{y} = X\beta + \varepsilon, \\ \begin{pmatrix} y_1 \\ \vdots \\ y_n \end{pmatrix} = \begin{pmatrix} x_{11} & \cdots & x_{1p} \\ \vdots & \ddots & \vdots \\ x_{n1} & \cdots & x_{np} \end{pmatrix} \begin{pmatrix} \beta_1 \\ \vdots \\ \beta_p \end{pmatrix} + \begin{pmatrix} \varepsilon_1 \\ \vdots \\ \varepsilon_n \end{pmatrix}. \end{gathered} \tag{3.7}$$

We then compute the least squares estimator $S(\beta)$ as:

$$\begin{aligned} S(\beta) &= \sum_{i=1}^{n} (y_i - \mathbf{x}_i^\intercal \beta)^2 \\ &= (\mathbf{y} - X\beta)^\intercal (\mathbf{y} - X\beta) \\ &= \mathbf{y}^\intercal \mathbf{y} - \mathbf{y}^\intercal X\beta - (X\beta)^\intercal \mathbf{y} + (X\beta)^\intercal (X\beta) \\ &= \mathbf{y}^\intercal \mathbf{y} - \mathbf{y}^\intercal X\beta - \beta^\intercal X^\intercal \mathbf{y} + \beta^\intercal X^\intercal X\beta, \end{aligned} \tag{3.8}$$

and seeing that the terms $\mathbf{y}^\intercal X\beta$ and $\beta^\intercal X^\intercal \mathbf{y}$ are scalars and one the transpose of the other, we can write:

$$= \mathbf{y}^\intercal \mathbf{y} - 2\beta^\intercal X^\intercal \mathbf{y} + \beta^\intercal X^\intercal X\beta. \tag{3.9}$$

Now we take the derivative with respect to β and set it to zero:

$$\begin{aligned} \frac{\partial S(\beta)}{\partial \beta} &= -2X^\intercal \mathbf{y} + 2X^\intercal X\beta = \mathbf{0} \\ &= -X^\intercal \mathbf{y} + X^\intercal X\beta = \mathbf{0} \\ &= (X^\intercal X)\beta = X^\intercal \mathbf{y}. \end{aligned} \tag{3.10}$$

These p equations in p parameters expressed in matrix form are called *normal equations*. When $X^\intercal X$ is invertible, we can solve for β as:

$$\beta = (X^\intercal X)^{-1} X^\intercal \mathbf{y}. \tag{3.11}$$

Bias and variance of $\hat{\beta}$ Under the assumption that the linear model generated the data:

- $$\mathrm{E}[\hat{\beta}] = \mathrm{E}[(X^\intercal X)^{-1} X^\intercal \mathbf{y}] = (X^\intercal X)^{-1} X^\intercal \,\mathrm{E}[\mathbf{y}] = \underbrace{(X^\intercal X)^{-1} X^\intercal X}_{\mathbb{1}} \beta = \beta. \tag{3.12}$$
- $$\begin{aligned} \mathrm{Var}(\hat{\beta}) &= \mathrm{Var}((X^\intercal X)^{-1} X^\intercal \mathbf{y}) \\ &= (X^\intercal X)^{-1} X^\intercal \sigma^2 \mathbb{1}_n X((X^\intercal X)^{-1})^\intercal && [\mathrm{Var}(A\mathbf{x}) = A\,\mathrm{Var}(\mathbf{x})A^\intercal] \\ &= (X^\intercal X)^{-1} X^\intercal \sigma^2 \mathbb{1}_n X(X^\intercal X)^{-1} && [((X^\intercal X)^{-1})^\intercal = (X^\intercal X)^{-1}] \\ &= \sigma^2 (X^\intercal X)^{-1} \underbrace{(X^\intercal X)(X^\intercal X)^{-1}}_{\mathbb{1}_p} \\ &= \sigma^2 \underbrace{(X^\intercal X)}_{p \times p}^{-1}. \end{aligned}$$

Bias-variance trade-off for linear models Also here assume that data are generated by a linear model. In earlier sections we defined:

$$\mathrm{E}[(y_0 - \hat{f}(\mathbf{x}_0))^2] = \underbrace{(f(\mathbf{x}_0) - \mathrm{E}[\hat{f}(\mathbf{x}_0)])}_{\text{Bias}}^2 + \mathrm{Var}(\hat{f}(\mathbf{x}_0)) + \mathrm{Var}(\varepsilon). \tag{3.13}$$

In the case of linear models, we can write:

Bias

$$\mathbf{x}_0^\intercal \beta - \mathrm{E}[\mathbf{x}_0^\intercal \hat{\beta}] = \mathbf{x}_0^\intercal \beta - \mathbf{x}_0^\intercal \underbrace{\mathrm{E}[\hat{\beta}]}_{\beta} = 0. \tag{3.14}$$

In other words, a zero bias in $\hat{\beta}$ results in a zero-biased model.

Variance

$$
\begin{aligned}
\text{(3.15)} \qquad \operatorname{Var}(\mathbf{x}_0^\intercal \hat{\beta}) &= \mathbf{x}_0^\intercal \operatorname{Var}(\hat{\beta}) \mathbf{x}_0 \\
&= \sigma^2 \operatorname{Trace}(\mathbf{x}_0^\intercal (X^\intercal X)^{-1} \mathbf{x}_0) \\
&= \sigma^2 \operatorname{Trace}((X^\intercal X)^{-1} \mathbf{x}_0 \mathbf{x}_0^\intercal) \qquad [\operatorname{Trace}(ABc) = \operatorname{Trace}(BCA)] \\
&= \sigma^2 \operatorname{Trace}\left(\left(\frac{X^\intercal X}{n} n\right)^{-1} \mathbf{x}_0 \mathbf{x}_0^\intercal\right).
\end{aligned}
$$

Assuming $\mathrm{E}[X_i] = 0$, $\operatorname{Var}(X_i) = 1$ and independence of X_is, then $\frac{X^\intercal X}{n} \approx \mathbb{1}_p$. Therefore:

$$
\text{(3.16)} \qquad \approx \sigma^2 \operatorname{Trace}((\mathbb{1}_p n)^{-1} \mathbb{1}_p) = \sigma^2 \operatorname{Trace}\left(\frac{1}{n} \mathbb{1}_p\right) = \frac{p}{n} \sigma^2.
$$

So, for a linear model:

$$
\text{(3.17)} \qquad \mathrm{E}[(y_0 - \hat{f}(\mathbf{x}_0))^2] \approx 0 + \frac{p}{n} \sigma^2 + \sigma^2,
$$

but we can only act on the second term. In fact, the prediction accuracy depends on the ratio $\frac{p}{n}$. In particular, we can distinguish three cases:

1. $n \gg p$: the bias is about zero and the variance is low, so the LR model will do very well in these cases (if the model is well-specified).
2. $n \approx p$: we have variability in $\hat{\beta}$ and, therefore, in the predictions. This leads to overfitting and poor generalization.
3. $n < p$: $X^\intercal X$ is not invertible, so there will be infinitely many β that minimize the least squares error, so infinitely many models that fit the data equally well.

Prediction accuracy of linear models Three main approaches to improve prediction accuracy of linear models are:

1. *Limiting p* by doing *variable selection*. In this way we reduce variance at the expense of some bias. We also achieve interpretability, as we can identify which predictors are relevant for the response.
2. *Shrinkage methods*, which replace the *least squares estimator* by alternative methods.
3. *Dimensionality reduction*, which projects the p predictors into a lower-dimensional space (with $M < p$) and use the M projections as predictors.

Subset selection The idea here is to identify a subset of important predictors among $X_1, \ldots, X_p$. *Ideally*, we should consider all possible subsets. The algorithm for selecting the best subset is the following:

1. Let M_0 be the null model (only intercept).
2. For $k = 1, \ldots, p$:
    a. Fit all $\binom{p}{k}$ models with k predictors.
    b. Pick the best among them, and call it M_k.

3. Select the best model out of $M_0, M_1, \ldots, M_p$ by using some model selection criterion (*e.g.*, AIC, BIC, CV).

The problem here is clear: with p predictors, we have 2^p possible subsets, so the number of models to fit grows exponentially with p.

Example. With $p = 2$ we only have 4 models to test. However, with $p = 100$ we have $2^{100} \approx 1.26 \cdot 10^{30}$ models to test, which is computationally infeasible. ◇

Therefore, we need to use some approximation methods to find a good subset of predictors. Three common approaches are called *stepwise methods*:

Forward selection

1. Let M_0 be the null model (only intercept).
2. For $k = 0, \ldots, p-1$:
   (a) Fit $p - k$ models that augment M_k with one of the remaining predictors.
   (b) Choose the best among them, and call it M_{k+1}.
   (c) Fix M_{k+1} and repeat.
3. Select the best model among $M_0, M_1, \ldots, M_p$ by using some model selection criterion (*e.g.*, AIC, BIC, CV).

With this method we reduce the number of models to fit from 2^p to $\frac{p(p+1)}{2} + 1$.

Backward selection The same procedure can be done in reverse, starting from the full model (if it's possible to fit it) and removing one predictor at a time.

Stepwise selection This method is a combination of the previous two. At each step we add a predictor like for forward selection and then we check if any of the previously included predictors can be removed without worsening the model fit. This variant is particularly useful when there are correlations among predictors.

3.3 Shrinkage methods

Some facts:

- Least squares estimator of β has no bias but high variance when $n \approx p$ and infinite variance when $n < p$.
- Improving prediction accuracy by reducing p (variable selection) can be computationally expensive.
- Alternatives to least squares estimator have been developed where β is estimated by adding a constraint or penalty on the squared-error objective function (likelihood).

Two popular approaches are:

1. Ridge regression,
2. Lasso regression.

3.3.1 Ridge regression

Recall that for the least squares estimator we have:

$$\hat{\beta}_{\text{LS}} = \underset{\beta}{\operatorname{argmin}} (\mathbf{Y} - X\beta)^\intercal (\mathbf{Y} - X\beta) \longrightarrow \hat{\beta}_{\text{LS}} = (X^\intercal X)^{-1} X^\intercal \mathbf{Y}. \tag{3.18}$$

In Ridge regression:

$$\hat{\beta}_{\text{Ridge}} = \underset{\beta}{\operatorname{argmin}} (\mathbf{Y} - X\beta)^\intercal (\mathbf{Y} - X\beta) \quad \text{s.t.} \sum_{j=1}^{p} \beta_j^2 \leqslant c, \quad c \geqslant 0. \tag{3.19}$$

If $c \to \infty$ we get the least squares estimator. As c decreases, more weight is put on the constraint, resulting in shrunk estimates.

The Ridge optimization problem can be rewritten as:

$$\hat{\beta}_{\text{Ridge}} = \underset{\beta}{\operatorname{argmin}} \max_{\lambda \geqslant 0} \left[(\mathbf{Y} - X\beta)^\intercal (\mathbf{Y} - X\beta) + \lambda \sum_{j=1}^{p} \beta_j^2 \right]. \tag{3.20}$$

This makes sense since:

- if we consider a β such that $\sum_{j=1}^{p} \beta_j^2 > c$, then we can set λ to a large value (∞) to make the objective function arbitrarily large, so β cannot be the solution of the optimization problem;
- if we consider a β such that $\sum_{j=1}^{p} \beta_j^2 < c$, then we can set λ to 0 to make the objective function as small as possible, so β must be the solution of the original optimization problem.

Here λ plays the role of c, just in the opposite way:

- if $\lambda \to 0$ we get the least squares estimator,
- as λ increases, more weight is put on the penalty, resulting in shrunk estimates.

The solution of the Ridge regression optimization problem is:

$$\begin{aligned}
\frac{\partial}{\partial \beta} \left[(\mathbf{Y} - X\beta)^\intercal (\mathbf{Y} - X\beta) + \lambda \sum_{j=1}^{p} \beta_j^2 \right] &= 0 \\
-2X^\intercal \mathbf{Y} + 2X^\intercal X\beta + 2\lambda\beta &= 0 \\
X^\intercal \mathbf{Y} - X^\intercal X\beta - \lambda\beta &= 0 \\
(X^\intercal X)\beta + \lambda \mathbb{1}\beta &= X^\intercal \mathbf{Y} \\
(X^\intercal X + \lambda \mathbb{1})\beta &= X^\intercal \mathbf{Y} \\
\hat{\beta}_{\text{Ridge}} &= (X^\intercal X + \lambda \mathbb{1})^{-1} X^\intercal \mathbf{Y}.
\end{aligned} \tag{3.21}$$

We can state some observations:

- This is a closed-form solution, as in the least squares case.
- It depends on λ.
- It is computationally efficient.
- For $\lambda > 0$, $X^\intercal X + \lambda \mathbb{1}$ is invertible, even if $n < p$.

Paths of solutions The larger the λ, the smaller the c and the more shrunk the estimates. Moreover, λ has a role in bias variance trade-off, so it is usually selected by cross-validation.

Bias of $\hat{\beta}_{\text{Ridge}}$

$$\begin{aligned}
\hat{\beta}_{\text{Ridge}} &= (X^\intercal X + \lambda \mathbb{1})^{-1} X^\intercal \mathbf{Y} \qquad [R = X^\intercal X] \\
&= (R + \lambda \mathbb{1})^{-1} X^\intercal \mathbf{Y} \\
&= (R + \lambda \mathbb{1})^{-1} R R^{-1} X^\intercal \mathbf{Y} \\
&= (R + \lambda \mathbb{1})^{-1} R \underbrace{(X^\intercal X)^{-1} X^\intercal \mathbf{Y}}_{\hat{\beta}_{\text{LS}}} \\
&= (R(\mathbb{1} + \lambda R^{-1}))^{-1} R \hat{\beta}_{\text{LS}} \\
&= (\mathbb{1} + \lambda R^{-1})^{-1} \underbrace{R^{-1} R}_{\mathbb{1}} \hat{\beta}_{\text{LS}} \\
&= \underbrace{(\mathbb{1} + \lambda (X^\intercal X)^{-1})^{-1}}_{A} \hat{\beta}_{\text{LS}}.
\end{aligned} \tag{3.22}$$

Therefore, $\hat{\beta}_{\text{Ridge}} = A\hat{\beta}_{\text{LS}}$ where A is a matrix that depends on λ and X.

We compute the expectation value of $\hat{\beta}_{\text{Ridge}}$:

$$\begin{aligned}
\mathrm{E}[\hat{\beta}_{\text{Ridge}}] &= A\, \mathrm{E}[\hat{\beta}_{\text{LS}}] \\
&= (\mathbb{1} + \lambda (X^\intercal X)^{-1})^{-1} \beta.
\end{aligned} \tag{3.23}$$

and the bias is:

$$\begin{aligned}
\mathrm{Bias}(\hat{\beta}_{\text{Ridge}}) &= \mathrm{E}[\hat{\beta}_{\text{Ridge}}] - \beta \\
&= (\mathbb{1} + \lambda (X^\intercal X)^{-1})^{-1} \beta - \beta \\
&= \left[(\mathbb{1} + \lambda (X^\intercal X)^{-1})^{-1} - \mathbb{1} \right] \beta.
\end{aligned} \tag{3.24}$$

We conclude that increasing λ increases the bias of $\hat{\beta}_{\text{Ridge}}$, but it can reduce the variance.

3.3.2 Lasso regression

The acronym means *Least Absolute Shrinkage and Selection Operator*. The optimization problem is:

$$\hat{\beta}_{\text{Lasso}} = \underset{\beta}{\operatorname{argmin}} (\mathbf{Y} - X\beta)^\intercal (\mathbf{Y} - X\beta) + \lambda \sum_{j=1}^{p} |\beta_j|, \tag{3.25}$$

with $\lambda \geqslant 0$ as tuning parameter. As before, this is equivalent to:

$$\hat{\beta}_{\text{Lasso}} = \underset{\beta}{\operatorname{argmin}} (\mathbf{Y} - X\beta)^\intercal (\mathbf{Y} - X\beta) \quad \text{s.t.} \sum_{j=1}^{p} |\beta_j| \leqslant c, \quad c \geqslant 0. \tag{3.26}$$

Role of λ Some facts about λ:

1. If $\lambda \to 0$ we get the least squares estimator.
2. As λ increases (c decreases), more weight is put on the penalty, resulting in shrunk estimates.
3. We get one $\hat{\beta}_{\text{Lasso}}$ for each λ, but there's no closed-form expression.
4. Efficient numerical algorithms have been developed to compute $\hat{\beta}_{\text{Lasso}}$ for a given λ.
5. λ is related to the bias-variance trade-off, so it is usually selected by cross-validation.
6. For λ sufficiently large, the Lasso estimates for some βs are shrunk exactly to zero, *i.e.*, we are doing variable selection.

Constrained optimization Now we ask a question: *why can Lasso estimate be exactly zero?* Let us consider just two predictors, so $p = 2$, and β_1 and β_2 are the two coefficients. The objective function is:

$$(\mathbf{Y} - X\beta)^\intercal (\mathbf{Y} - X\beta). \tag{3.27}$$

The two additional terms in Ridge and Lasso regression are different when interpreted as norms:

- *Ridge regression* term is the ℓ_2 norm of β, meaning that it represents the distance of β from the origin in the p-dimensional space. The constraint $\sum_{j=1}^{p} \beta_j^2 \leqslant c$ defines a sphere in this space, and the solution is found at the point where the contours of the least squares objective function touch this sphere.
- *Lasso regression* term is the ℓ_1 norm of β, meaning that it represents the sum of the absolute values of the coefficients. The constraint $\sum_{j=1}^{p} |\beta_j| \leqslant c$ defines a diamond-shaped region in the p-dimensional space, and the solution is found at the point where the contours of the least squares objective function touch this diamond. The corners of the diamond correspond to points where one or more coefficients are exactly zero, which is why Lasso can produce sparse solutions.

Example. Let's assume that we have only one predictor ($p = 1$), then:

$$\hat{\beta}_{\text{Lasso}} = \underset{\beta}{\operatorname{argmin}} (\mathbf{Y} - \mathbf{X}\beta)^\intercal (\mathbf{Y} - \mathbf{X}\beta) + \lambda |\beta|. \tag{3.28}$$

If we take the derivative with respect to β and set it to zero, we get:

$$-2\mathbf{X}^\intercal (\mathbf{Y} - \mathbf{X}\beta) + \lambda \operatorname{sgn}(\beta) = 0, \tag{3.29}$$

where $\operatorname{sgn}(\beta)$ is a variant of the sign function defined as:

$$\operatorname{sgn}(\beta) = \begin{cases} 1 & \text{if } \beta > 0, \\ [-1, 1] & \text{if } \beta = 0, \\ -1 & \text{if } \beta < 0. \end{cases} \tag{3.30}$$

Here we used the concept of *subdifferential*, which is a generalization of the derivative for functions that are not differentiable. ◇

$\hat{\beta}_{\text{Lasso}}$ vs $\hat{\beta}_{\text{LS}}$ Consider the least squares estimator $\hat{\beta}_{\text{LS}}$ for one predictor, which is the solution of the optimization problem:

$$\mathbf{X}^\intercal (\mathbf{Y} - \mathbf{X}\beta) = 0. \tag{3.31}$$

We distinguish three cases:

- $$\begin{aligned} \beta_{\text{Lasso}} > 0 &\iff \mathbf{X}^\intercal (\mathbf{Y} - \mathbf{X}\beta_{\text{Lasso}}) = \frac{\lambda}{2} \\ &\iff \mathbf{X}^\intercal \mathbf{Y} = \mathbf{X}^\intercal \mathbf{X}\beta_{\text{Lasso}} + \frac{\lambda}{2} \\ &\iff \beta_{\text{Lasso}} = (\mathbf{X}^\intercal \mathbf{X})^{-1} \mathbf{X}^\intercal \mathbf{Y} - \frac{\lambda}{2} (\mathbf{X}^\intercal \mathbf{X})^{-1} \\ &\iff \beta_{\text{Lasso}} = \beta_{\text{LS}} - \frac{\lambda}{2} (\mathbf{X}^\intercal \mathbf{X})^{-1}. \end{aligned}$$

- $\beta_{\text{Lasso}} < 0 \iff \mathbf{X}^\intercal(\mathbf{Y} - \mathbf{X}\beta_{\text{Lasso}}) = -\frac{\lambda}{2} \iff \beta_{\text{Lasso}} = \beta_{\text{LS}} + \frac{\lambda}{2}(\mathbf{X}^\intercal\mathbf{X})^{-1}$.
- $\beta_{\text{Lasso}} = 0 \iff \mathbf{X}^\intercal(\mathbf{Y} - \mathbf{X}\beta_{\text{Lasso}}) \in \left[-\frac{\lambda}{2}, \frac{\lambda}{2}\right]$

$$\iff \mathbf{X}^\intercal\mathbf{Y} - \underbrace{(\mathbf{X}^\intercal\mathbf{X})\beta_{\text{Lasso}}}_{0} \in \left[-\frac{\lambda}{2}, \frac{\lambda}{2}\right]$$

$$\iff \beta_{\text{LS}} = (\mathbf{X}^\intercal\mathbf{X})^{-1}\mathbf{X}^\intercal\mathbf{Y} \in \left[-\frac{\lambda}{2}(\mathbf{X}^\intercal\mathbf{X})^{-1}, \frac{\lambda}{2}(\mathbf{X}^\intercal\mathbf{X})^{-1}\right].$$

Soft-thresholding operator The soft-thresholding operator is used as the basis of the coordinate descent algorithm for solving the Lasso optimization problem when $p > 1$.

4 Tree based methods

The main idea behind these methods is to divide the space into simple regions. The predicted outcome, $\hat{y}$ or class, depends on the region of the space an observation falls into. The partitioning is the result of some sequential splitting rules, and this is the reason why this method is called *decision tree*.

4.1 Regression trees

Given a tree with J regions $R_1, R_2, \ldots, R_J$, then:

Regression Given a new observation $\mathbf{x}$, consider the region of the space it falls into and predict y with the average response of all the training observations in that region.

Classification Given a new observation $\mathbf{x}$, consider the region R_j it falls into and make a prediction based on:

$$\hat{\mathrm{P}}(c \mid \mathbf{x}) = \hat{\mathrm{P}}(c \mid R_j), \tag{4.1}$$

i.e., the proportion of training observations in the region R_j that belong to class c, $c = 1, 2, \ldots, K$. Then, we set a threshold on these probabilities as before.

How to build a tree The goal here is to divide the space into "boxes". Two key aspects to keep in mind are:

- The choice of optimality criterion to decide the best split at each step.
- Devising a computationally efficient procedure, since it is infeasible to look at all possible partitions.

Choice of optimality criterion

Regression In the case of regression, the natural choice under a squared error loss is to find regions $R_1, \ldots, R_j$ that minimize:

$$\mathrm{RSS} = \sum_{j=1}^{J} \sum_{i \in R_j} (y_i - \hat{y}_{R_j})^2 \tag{4.2}$$

(Residual Sum of Squares), where $\hat{y}_{R_j}$ is the average of the response values for the training observations in R_j.

Classification The most natural one in this setting is the following:

$$E = \sum_{j=1}^{J} E_j, \qquad E_j = 1 - \max_c \hat{\mathrm{P}}(c \mid R_j), \quad j = 1, 2, \ldots, J. \tag{4.3}$$

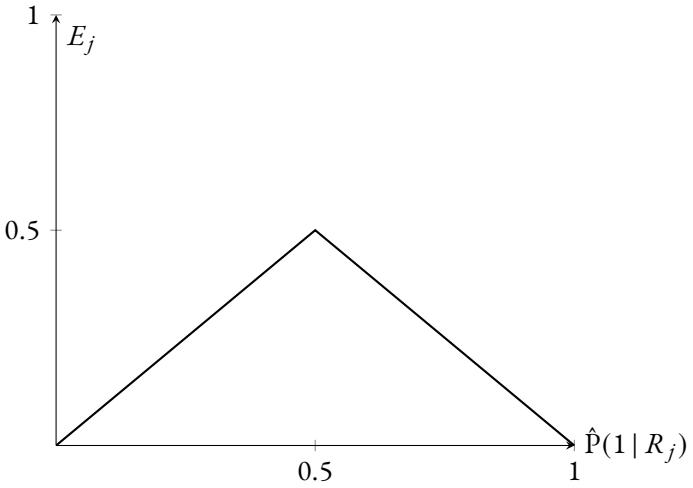

Figure 4.1 *Misclassification error as a function of* $\hat{P}$.

However, this function is not "smooth", so it is not ideal for optimization. In fact, with two classes, we have the following probabilities:

$$\hat{P}(1 \mid R_j) \qquad 1 - \hat{P}(1 \mid R_j) \tag{4.4}$$

and the error function is:

$$\begin{aligned} E_j &= 1 - \max\{\hat{P}(1 \mid R_j), 1 - \hat{P}(1 \mid R_j)\} \\ &= 1 - \max\{\hat{P}, 1 - \hat{P}\}, \end{aligned} \tag{4.5}$$

which is plotted in Figure 4.1. Two smooth alternatives are:

1. *Gini index*:

$$G = \sum_{j=1}^{J} G_j, \quad G_j = \sum_{c=1}^{K} \hat{P}(c \mid R_j)(1 - \hat{P}(c \mid R_j)), \tag{4.6}$$

2. *Entropy* or *deviance*:

$$D = \sum_{j=1}^{J} D_j, \quad D_j = -\sum_{c=1}^{K} \hat{P}(c \mid R_j) \log \hat{P}(c \mid R_j). \tag{4.7}$$

The next step is to find the partitioning that minimizes the chosen optimality criterion. However, this search would be computationally expensive if we look at all possible partitions. We need some more efficient procedures.

4.1.1 Recursive binary splitting

This method has the following properties:

- *Top-down approach*: we start with one region and we split it.
- *Binary*: at each step, we split one region into two.

- *Greedy*: at each step, we look for the best split at that step, without checking ahead.

The algorithm consists of the following steps:

1. For each predictor X_k and all its cutpoints s, define two regions:

$$R_-(k,s) = \{\mathbf{x} \mid X_k < s\}, \qquad R_+(k,s) = \{\mathbf{x} \mid X_k \geqslant s\}. \tag{4.8}$$

   In practice, the cutpoints are chosen:

   - For continuous predictors, as a discrete sequence.
   - For categorical variables, as all the possible splits of the categories into two groups.

2. Calculate the optimality criterion for each (k,s) partitioning and choose optimal partition. This gives us two initial regions R_1 and R_2.
3. Repeat the procedure in each region and choose the best partitioning according to the optimality criterion.
4. Continue until a stopping criterion is satisfied, *e.g.*, a maximal number of observations per region is reached, or the optimality criterion is not sufficiently reduced.

Big issue with this method The models fitted in this way have a too high variance (overfitting). Some solution to this problem are:

1. Fit a smaller size of the tree. This reduces the variance, but it increases the bias.
2. Stop dividing when reduction in RSS/Gini/entropy is below some threshold.

   Note. Actually this solution introduces another potential problem, which is that a "unimportant" split early on could result in a good split later on. ○

3. Build a big tree and prune it back, *i.e.*, remove branches to get a smaller subtree. This is the most common solution, and it is implemented in the *cost-complexity algorithm*:

   (a) Fix an hyperparameter $\alpha \geqslant 0$.

   (b) Use recursive binary splitting to build a large tree T_0, with a small number of observations in each region.

   (c) Choose the subtree $T_\alpha \subset T_0$ such that:

$$T_\alpha = \operatorname*{argmin}_{T \subset T_0} \sum_{j=1}^{|T|} \mathrm{OC}(R_j) + \alpha|T|, \tag{4.9}$$

   where the cardinality of the tree $|T|$ is the number of terminal nodes, and $\mathrm{OC}(R_j)$ is the chosen optimality criterion for region R_j.

If $\alpha = 0$, then $T_\alpha = T_0$, while as α increases, T_α gets smaller and smaller, in a nested fashion (pruning).

4.1.2 Cross-validation

We cannot select α by looking at the training error, but rather on some unseen data (test-set or cross-validation). In practice, doing cross-validation:

1. We choose a sequence of values of α.

2. For each α, we fit the tree on $k-1$ folds and we prune it.
3. We then use T_α to make predictions on the left-out fold.
4. We compute the CV error.

4.2 Bagging

Decision trees are easily interpretable but they tend to have a high variance. Bagging and boosting are two techniques that reduce the variance of a statistical learning method without increasing the bias. The main disadvantage is the loss of interpretability, but there are various techniques to overcome this problem.

The term stands for *Bootstrap aggregation*. The motivation behind this method is the following: assume we have n independent random variables $Z_1, \dots, Z_n$ with all the same variance σ^2 and mean μ. Consider $\overline{Z} = \frac{1}{n}\sum_{i=1}^n Z_i$. Then:

$$\begin{aligned} \operatorname{Var}(\overline{Z}) &= \operatorname{Var}\left(\frac{1}{n}\sum_{i=1}^n Z_i\right) = \frac{1}{n^2}\sum_{i=1}^n \operatorname{Var}(Z_i) = \frac{\sigma^2}{n}, \\ \mathrm{E}[\overline{Z}] &= \mathrm{E}\left[\frac{1}{n}\sum_{i=1}^n Z_i\right] = \frac{1}{n}\sum_{i=1}^n \mathrm{E}[Z_i] = \mu, \end{aligned} \tag{4.10}$$

where the summation is taken outside of the variance because the Z_i are independent. We notice that the variance of the aggregated variable $\overline{Z}$ is smaller than the variance of each Z_i by a factor of n, while the mean is unchanged. In out context, consider B independent training sets. We could then build B models $\hat{f}_1, \dots, \hat{f}_B$ and then average them to reduce variance:

$$\hat{f}_{\text{Avg}}(\mathbf{x}) = \frac{1}{B}\sum_{i=1}^B \hat{f}_i(\mathbf{x}). \tag{4.11}$$

The problem is that we normally have only one dataset. The solution is to use the *bootstrap*: build B training sets of the same size as the original dataset by repeatedly sampling with replacement from the dataset that we have, *i.e.*, take one row, put it into the new dataset, and then put it back into the original dataset. This way, we can build a model on each of the B boostrapped datasets, $\hat{f}_i^*$ and then average the predictions:

$$\hat{f}_{\text{Bag}}(\mathbf{x}) = \frac{1}{B}\sum_{i=1}^B \hat{f}_i^*(\mathbf{x}). \tag{4.12}$$

Some remarks:

1. For classification, $\hat{f}_i^*$ can be taken either as the estimated probabilities $\hat{\mathrm{P}}_i^*(c \mid \mathbf{x})$ for each class or as the predicted class for each tree (after setting a threshold). In the second case, you take the majority vote as the predicted class.
2. The B bootstrapped datasets will not be independent so we do not get a full reduction in variance by a factor of B.
3. Bagging reduces variance without increasing bias. Therefore, it makes sense to do bagging on complex models so the starting bias is low and we benefit from the variance reduction.
4. B should be taken as large as computationally possible (no issues with overfitting).

Out-of-Bag error estimation Having built a model by bagging, we want to test its prediction accuracy on unseen data. We can do cross-validation, but it is computationally expensive. An efficient alternative is to make use of the bootstrap resampling.

Note. Each bootstrapped dataset has n observations. Consider the observation $\mathbf{x}_i$ from the original dataset. What is the probability that $\mathbf{x}_i$ is not included in a bootstrapped dataset?

$$\left(1 - \frac{1}{n}\right)^n \xrightarrow{n\to\infty} e^{-1} \approx 0.368. \tag{4.13}$$

That is, each sample uses approximately 63% of the data. We call the unseen observation *out-of-bag observations* and we can evaluate the model on them. ○

For each observation $\mathbf{x}$, make a prediction based on the trees that have been fitted on samples that did not include $\mathbf{x}$. An average (or *consensus class*) is taken among these predictions only. Errors, ROC curves, etc. are built using these probabilities.

Cost of Bagging

1. Loss of interpretability: an average of a tree is not a tree. However, one can calculate various measures of variable importance. For each predictor, add up the total amount that the RSS/Gini/Entropy is decreased by splits on that predictor across the B trees.
2. Bootstrap samples are correlated and not independent: the variance of the sum of correlated variables is higher than the variance of a sum of independent variables.

Random forests provide a way to "decorrelate" trees.

4.3 Random forests

Question: why would the trees grown in bagging be correlated? Strong predictors will tend to appear at the first splits of the trees and the trees will then look quite similar. The idea is, before each split in each tree, draw m features at random from the p predictors and look for splits among these variables only.

If there is a strong predictor, it will now appear at first split only on m/p trees out of the B that are grown. Therefore, m has now a role in the bias-variance tradeoff:

- Strong predictors not included will lead to higher bias;
- Strong predictors always included will lead to higher variance.

Choice of m

1. $m = p$ is bagging, so random forests is a generalization of bagging.
2. Typical default values are:

   Classification $m = \lfloor \sqrt{p} \rfloor$.

   Regression $m = \max\{\lfloor p/3 \rfloor, 1\}$.

3. The best strategy is to use cross-validation to find the optimal trade-off on any given dataset. Indeed, m is also related to the number of strong predictors in the dataset: many strong predictors will require a small m, while few strong predictors will require a large m.

4.4 Boosting

It is based on the idea of aggregating trees that are sequentially grown from the same data. Each tree is fitted on the "residuals" of the previous model. We use *weak learners* (simple models) and update these slowly by fitting a tree to the residuals of the previous model.

An algorithm for boosting is called *Gradient Boosting*:

1. Let $\mathrm{L}(y_i, \hat{y}_i) = (y_i - \hat{y}_i)^2$ be the loss function.
2. Initialize the model with $f_0(\mathbf{x}) = \operatorname{argmin}_\gamma \sum_{i=1}^n \mathrm{L}(y_i, \gamma)$. For the squared error loss, this is just the mean of the y_i, $\bar{y}$.
3. For $m = 1, \ldots, M$:
   a. Calculate $r_{im} = y_i - f_{m-1}(\mathbf{x}_i)$ for $i = 1, \ldots, n$.
   b. Fit a tree to the residuals, giving terminal regions R_{jm}, $j = 1, \ldots, J_m$.
   c. For $j = 1, \ldots, J_m$, compute:

   $$\gamma_{jm} = \operatorname*{argmin}_\gamma \sum_{\mathbf{x}_i \in R_{jm}} \mathrm{L}(y_i - f_{m-1}(\mathbf{x}_i), \gamma), \tag{4.14}$$

   which can be interpreted as the average of the residuals in the j-th region of the m-th tree.
   d. Update:

   $$f_m(\mathbf{x}_i) = f_{m-1}(\mathbf{x}_i) + \sum_{j=1}^{J_m} \gamma_{jm} \mathbb{1}(\mathbf{x}_i \in R_{jm}). \tag{4.15}$$

4. Output $\hat{f}(\mathbf{x}) = f_M(\mathbf{x})$.

Some remarks:

1. J_m is typically small (2, 3).
2. M is related to the bias-variance tradeoff, so it needs to be tuned by cross-validation or similar.
3. A common version of boosting has an additional parameter, called *learning rate*, $0 < \nu \leqslant 1$, that is inserted in the update step:

   $$f_m(\mathbf{x}_i) = f_{m-1}(\mathbf{x}_i) + \nu \sum_{j=1}^{J_m} \gamma_{jm} \mathbb{1}(\mathbf{x}_i \in R_{jm}). \tag{4.16}$$

   Usually, ν is set to a small value and M is tuned.
4. An extension of the algorithm is *Stochastic Gradient Boosting*, where it fits a tree on a random sample of the data. Therefore, one can evaluate the performance of the method during the learning.

5 Support Vector Machines

All the methods analyzed so far try to approximate the quantity $\mathrm{P}(1 \mid \mathbf{x})$. SVMs concentrate their effects on finding a function $g(\mathbf{x})$ that best separate the two classes. We can identify three versions of SVMs:

1. *Maximal Margin Classifiers*, used when the data are linearly separable;
2. *Support Vector Classifiers*, used when the data are not linearly separable;
3. *Support Vector Machines*, an extension to non-linear classifiers.

5.1 Maximal Margin Classifiers

When we say that data is linearly separable, we can actually identify an infinite number of hyperplanes that can separate the two classes. Therefore, the problem, after having identified them, is to pick one based on some optimality criterion.

Definition 2 (Hyperplane). *In a p-dimensional space, a hyperplane is a flat affine subspace of dimension $p-1$.*

In general, a hyperplane is identified by the equation:

$$\beta_0 + \beta_1 X_1 + \cdots + \beta_p X_p = 0 \iff \beta_0 + \boldsymbol{\beta}^\intercal \mathbf{x} = 0, \quad \text{with } \boldsymbol{\beta} = \begin{pmatrix} \beta_1 \\ \vdots \\ \beta_p \end{pmatrix}. \tag{5.1}$$

The hyperplane divides the space into two half-spaces, one where $\beta_0 + \boldsymbol{\beta}^\intercal \mathbf{x} > 0$ and one where $\beta_0 + \boldsymbol{\beta}^\intercal \mathbf{x} < 0$. The points that lie on the hyperplane satisfy $\beta_0 + \boldsymbol{\beta}^\intercal \mathbf{x} = 0$.

Separating hyperplane Given the data:

$$X_{n \times p}, \mathbf{x}_1, \ldots, \mathbf{x}_n \tag{5.2}$$

the goal is to produce the response variables $y_i \in \{-1, 1\}$. We map y_i to 1 if $\beta_0 + \boldsymbol{\beta}^\intercal \mathbf{x}_i > 0$ and to -1 if $\beta_0 + \boldsymbol{\beta}^\intercal \mathbf{x}_i < 0$. We can also write this in a more compact way as:

$$y_i(\beta_0 + \boldsymbol{\beta}^\intercal \mathbf{x}_i) > 0, \quad i = 1, \ldots, n. \tag{5.3}$$

Such a hyperplane can be used for classification for a new $\mathbf{x}^*$:

$$f(\mathbf{x}^*) = \beta_0 + \boldsymbol{\beta}^\intercal \mathbf{x}^* \to y^* = \begin{cases} 1 & \text{if } f(\mathbf{x}^*) > 0, \\ -1 & \text{if } f(\mathbf{x}^*) < 0. \end{cases} \tag{5.4}$$

In this context, the *optimal* separating hyperplane is the one that maximizes the margin, where the *margin* is the minimum distance of the points to the hyperplane.

A hyperplane is defined by its orthogonal direction given by β. Let us consider only βs such that $\|\beta\| = \sqrt{\beta_1^2 + \cdots + \beta_p^2} = 1$. The distance of a point $\mathbf{x}_i$ to the hyperplane is obtained considering a generic point $\mathbf{z}$ on the hyperplane, *i.e.*, such that $\beta_0 + \beta^\top \mathbf{z} = 0$, and projecting the vector $\mathbf{x}_i - \mathbf{z}$ on the direction β:

$$\operatorname{dist}(\mathbf{x}, H) = \left|\beta^\top (\mathbf{x} - \mathbf{z})\right| = \left|\beta^\top \mathbf{x} - \underbrace{\beta^\top \mathbf{z}}_{\beta_0 + \beta^\top \mathbf{z} = 0}\right| = \left|\beta_0 + \beta^\top \mathbf{x}\right|. \tag{5.5}$$

Then, the maximal margin classifier is defined by:

$$\begin{aligned} (\hat{\beta}_0, \hat{\beta}) = \underset{\beta_0, \beta}{\operatorname{argmin}} \min_{i=1,\ldots,n} & y_i(\beta_0 + \beta^\top \mathbf{x}_i) \\ \text{s.t. } & \|\beta\| = 1. \end{aligned} \tag{5.6}$$

Equivalently, we can write:

$$\begin{aligned} & \underset{\beta_0, \ldots, \beta_p}{\operatorname{argmax}} M \\ & \text{s.t. } \beta_1^2 + \cdots + \beta_p^2 = 1, \\ & \quad y_i(\beta_0 + \beta^\top \mathbf{x}_i) \geqslant M, \quad i = 1, \ldots, n. \end{aligned} \tag{5.7}$$

Some remarks:

- We need at least two observations that are closest to the hyperplane and on either side of the hyperplane.
- In general, there are not many observations that are the closest to the hyperplane. They are called support vectors and completely characterize the hyperplane.

However, there are also some problems:

1. Classes are typically not linearly separable.
2. The classifier is sensitive to a small number of support vectors: this could lead to overfitting and it is not robust to outliers.

A solution is to relax the conditions, *i.e.*, to allow some violations. From *hard margin* we move to *soft margin* classifiers, which are also called *support vector classifiers*.

5.2 Support Vector Classifiers

The optimization problem now becomes:

$$\begin{aligned} & \max_{\beta_0, \ldots, \beta_p} M \\ & \text{s.t. } \|\beta\| = \beta_1^2 + \cdots + \beta_p^2 = 1, \\ & \quad y_i(\beta_0 + \beta^\top \mathbf{x}_i) \geqslant M(1 - \varepsilon_i), \quad i = 1, \ldots, n, \\ & \quad \varepsilon_i \geqslant 0, \quad i = 1, \ldots, n, \\ & \quad \sum_{i=1}^{n} \varepsilon_i \leqslant C, \end{aligned} \tag{5.8}$$

where ε_i are called *slack variables* and C is a tuning parameter that controls the amount of violation allowed.

Slack variables Based on where a point is with respect to the hyperplane, we can identify three cases:

- $\varepsilon_i = 0$ for observations on the correct side of the hyperplane and at least M away from it;
- $0 < \varepsilon_i < 1$ for observations on the correct side of the hyperplane but less than M away from it;
- $\varepsilon_i > 1$ for observations on the wrong side of the hyperplane.

Choice of C If we set $C = 0$, we get the maximal margin classifier, *i.e.*, we admit no errors, if the two classes are linearly separable, otherwise the optimization problem has no solution. If we set $C > 0$, we can consider it as an upper bound on the number of violations. C is also connected to the bias-variance trade-off:

- A small C gives rise to a small margin and few violations, which reduces the number of support vectors and leads to low bias and high variance.
- A large C gives rise to many violations with a large margin, together with high bias and low variance.

Since C needs to be tuned, methods like CV or similar should be used.

Now let's analyze practical ways of solving the optimization problem. The first constraint, *i.e.*:

$$C_1 : \|\beta\| = \beta_1^2 + \cdots + \beta_p^2 = 1, \tag{5.9}$$

is not very good to have, since it is not convex.

Example. Consider $p = 2$. Then, the constraint becomes:

$$\beta_1^2 + \beta_2^2 = 1, \tag{5.10}$$

which is a circumference. The set of points that satisfy this constraint is not convex, since the segment connecting two points on the circumference does not necessarily lie on the circumference itself. ◇

We can get rid of this constraint by replacing the second constraint, *i.e.*:

$$\begin{aligned} \tilde{C}_2 : \frac{1}{\|\beta\|} y_i(\beta_0 + \beta^\intercal \mathbf{x}_i) &\geqslant M(1 - \varepsilon_i), \quad i = 1, \dots, n, \\ \iff y_i(\beta_0 + \beta^\intercal \mathbf{x}_i) &\geqslant \|\beta\| M(1 - \varepsilon_i), \quad i = 1, \dots, n. \end{aligned} \tag{5.11}$$

Now, among all the βs, we consider the ones such that $\|\beta\| = 1/M$, *i.e.*, $M = 1/\|\beta\|$. Therefore, the

equivalent optimization problem is:

$$
\begin{aligned}
&\max_{\beta_0, \beta, \varepsilon} \frac{1}{\|\beta\|} \\
&\text{s.t. } y_i(\beta_0 + \beta^\top \mathbf{x}_i) \geqslant 1 - \varepsilon_i, \quad i = 1, \ldots, n, \\
&\quad \varepsilon_i \geqslant 0, \quad i = 1, \ldots, n, \\
&\quad \sum_{i=1}^{n} \varepsilon_i \leqslant C \\
&\Updownarrow \\
&\min_{\beta_0, \beta, \varepsilon} \|\beta\|^2 \\
&\text{s.t. } y_i(\beta_0 + \beta^\top \mathbf{x}_i) \geqslant 1 - \varepsilon_i, \quad i = 1, \ldots, n, \\
&\quad \varepsilon_i \geqslant 0, \quad i = 1, \ldots, n, \\
&\quad \sum_{i=1}^{n} \varepsilon_i \leqslant C.
\end{aligned}
\tag{5.12}
$$

In both cases, obviously, we have n constraints coming from the second line, n constraints coming from the third line and one constraint coming from the fourth line, for a total of $2n + 1$ constraints. In the second formulation, the objective function is convex and the constraints are linear, so we are sure that there is one and only one global minimum.

Optimization with equality and inequality constraints Consider:

$$
\begin{aligned}
&\min_{\alpha} f(\alpha) \\
&\text{s.t. } g_i(\alpha) = 0, \quad i = 1, \ldots, d, \\
&\quad h_j(\alpha) \leqslant 0, \quad j = 1, \ldots, m.
\end{aligned}
\tag{5.13}
$$

The solution to this problem comes from the *generalized Lagrangian*, which is defined as:

$$
\mathcal{L}(\alpha, \gamma, \delta) = f(\alpha) + \sum_{i=1}^{d} \gamma_i h_i(\alpha) + \sum_{j=1}^{m} \delta_j g_j(\alpha), \tag{5.14}
$$

where $\gamma = (\gamma_1, \ldots, \gamma_d)$ and $\delta = (\delta_1, \ldots, \delta_m)$ are the Lagrange multipliers. To justify the use of this function, consider this quantity:

$$
\theta_{\text{P}}(\alpha) = \max_{\substack{\gamma, \delta \\ \delta \geqslant 0}} \mathcal{L}(\alpha, \gamma, \delta) = \mathcal{L}(\alpha, \gamma^*, \delta^*). \tag{5.15}
$$

Now we can link:

$$
\begin{aligned}
\min_{\alpha} f(\alpha) &= \min_{\alpha} \theta_{\text{P}}(\alpha) \\
&= \min_{\alpha} \max_{\substack{\gamma, \delta \\ \delta \geqslant 0}} \mathcal{L}(\alpha, \gamma, \delta).
\end{aligned}
\tag{5.16}
$$

Note. Why this? Consider an α that does not satisfy the constraints, *i.e.*, an α such that $h_i(\alpha) \neq 0$ or $g_j(\alpha) > 0$. Then, $\max_{\gamma, \delta} \mathcal{L}(\alpha, \gamma, \delta) = +\infty$, since we can make γ_i or δ_j arbitrarily large. We conclude that α will not be the solution to the optimization problem. ○

The p in θ_{P} stands for *primal*, since we are still considering the original optimization problem. We can also define the *dual* version of the problem as:

$$\max_{\substack{\gamma,\delta \\ \delta \geqslant 0}} \theta_{\text{D}}(\gamma, \delta) = \max_{\substack{\gamma,\delta \\ \delta \geqslant 0}} \min_{\alpha} \mathcal{L}(\alpha, \gamma, \delta). \tag{5.17}$$

Note. In general, the two problems are not equivalent. In particular,

$$\text{max-min} \leqslant \text{min-max}. \tag{5.18}$$

However, here, *i.e.*, with a convex objective function f, convex constraints g_i and linear functions h_i, they are equivalent. ○

Karush-Kuhn-Tucker conditions Consider α^*, γ^* and δ^* at the optimum and $\theta_{\text{P}}(\alpha^*) = \theta_{\text{D}}(\gamma^*, \delta^*)$. Then:

$$\delta_i^* g_i(\alpha^*) = 0, \quad i = 1, \dots, m. \quad \text{(Complementary slackness conditions)} \tag{5.19}$$

Proof.

$$\begin{aligned} f(\alpha^*) = \min_{\alpha} \mathcal{L}(\alpha, \gamma^*, \delta^*) &= \min_{\alpha} \left(f(\alpha) + \sum_{i=1}^{d} \gamma_i^* h_i(\alpha) + \sum_{j=1}^{m} \delta_j^* g_j(\alpha) \right) \\ &\leqslant f(\alpha^*) + \underbrace{\sum_{i=1}^{d} \gamma_i^* h_i(\alpha^*)}_{0} + \underbrace{\sum_{j=1}^{m} \delta_j^* g_j(\alpha^*)}_{\leqslant 0} \\ &\leqslant f(\alpha^*). \end{aligned} \tag{5.20}$$

This implies:

$$\sum_{i=1}^{m} \underbrace{\delta_i^* g_i(\alpha^*)}_{\leqslant 0} = 0 \implies \delta_i^* g_i(\alpha^*) = 0, \quad i = 1, \dots, m. \tag{5.21}$$

□

Back to Support Vector Classifiers Now, we need to rewrite the constraints to apply the KKT conditions. We have:

$$\begin{aligned} y_i(\beta_0 + \beta^\top \mathbf{x}_i) \geqslant 1 - \varepsilon_i &\iff -y_i(\beta_0 + \beta^\top \mathbf{x}_i) + 1 - \varepsilon_i \leqslant 0, \quad i = 1, \dots, n, \\ \varepsilon_i \geqslant 0 &\iff -\varepsilon_i \leqslant 0, \quad i = 1, \dots, n, \\ \sum_{i=1}^{n} \varepsilon_i \leqslant C &\iff \sum_{i=1}^{n} \varepsilon_i - C \leqslant 0. \end{aligned} \tag{5.22}$$

The Lagrangian is:

$$\begin{aligned} \mathcal{L}(\beta_0, \beta, \varepsilon, \gamma, \delta, \mu) = &\frac{1}{2}\|\beta\|^2 \\ &- \sum_{i=1}^{n} \gamma_i (y_i(\beta_0 + \beta^\top \mathbf{x}_i) - 1 + \varepsilon_i) \\ &- \sum_{i=1}^{n} \delta_i \varepsilon_i \\ &+ \mu \left(\sum_{i=1}^{n} \varepsilon_i - C \right). \end{aligned} \tag{5.23}$$

Then:

$$\min_{\beta_0, \beta, \varepsilon} \frac{1}{2}\|\beta\|^2 = \underbrace{\min_{\beta_0, \beta, \varepsilon} \max_{\gamma, \delta, \mu \geqslant 0} \mathcal{L}(\cdot)}_{\text{Primal}} = \underbrace{\max_{\gamma, \delta, \mu \geqslant 0} \min_{\beta_0, \beta, \varepsilon} \mathcal{L}(\cdot)}_{\text{Dual}}. \tag{5.24}$$
$$\text{s.t. } \ldots$$

To optimize the function we need to compute the derivatives with respect to β_0, β and ε and set them equal to zero. We have:

$$\frac{\partial \mathcal{L}}{\partial \beta_0} = -\sum_{i=1}^{n} \gamma_i y_i = 0 \implies \sum_{i=1}^{n} \gamma_i y_i = 0, \tag{5.25}$$

$$\frac{\partial \mathcal{L}}{\partial \beta} = \beta - \sum_{i=1}^{n} \gamma_i y_i \mathbf{x}_i = 0 \implies \beta = \sum_{i=1}^{n} \gamma_i y_i \mathbf{x}_i, \tag{5.26}$$

$$\frac{\partial \mathcal{L}}{\partial \varepsilon_i} = \mu - \gamma_i - \delta_i = 0 \implies \gamma_i = \mu - \delta_i, \quad i = 1, \ldots, n. \tag{5.27}$$

If we plug these values back into the Lagrangian, we get:

$$\begin{aligned}
\theta_{\mathrm{D}}(\gamma, \delta, \mu) &= \frac{1}{2}\beta^{\mathsf{T}}\beta - \sum_{i=1}^{n}(\underbrace{\gamma_i y_i}_{0}\beta_0 + \gamma_i y_i \beta^{\mathsf{T}}\mathbf{x}_i - \gamma_i + \underbrace{\varepsilon_i \gamma_i}) - \sum_{i=1}^{n}\delta_i\varepsilon_i - \mu\sum_{i=1}^{n}\varepsilon_i - \mu C \\
&\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad -\gamma_i - \delta_i + \mu = 0 \\
&= \frac{1}{2}\beta^{\mathsf{T}}\beta - \beta^{\mathsf{T}}\underbrace{\sum_{i=1}^{n}\gamma_i y_i \mathbf{x}_i}_{\beta} + \sum_{i=1}^{n}\gamma_i - \mu C \qquad (5.28) \\
&= \sum_{i=1}^{n}\gamma_i - \frac{1}{2}\sum_{i,j}\gamma_i\gamma_j y_i y_j \mathbf{x}_i \cdot \mathbf{x}_j - \mu C.
\end{aligned}$$

Finally, the dual problem is:

$$\begin{aligned}
&\max_{\gamma, \mu} \sum_{i=1}^{n}\gamma_i - \frac{1}{2}\sum_{i,j}\gamma_i\gamma_j y_i y_j \mathbf{x}_i \cdot \mathbf{x}_j - \mu C, \\
&\text{s.t. } 0 \leqslant \gamma_i \leqslant \mu, \quad i = 1, \ldots, n, \qquad (5.29) \\
&\sum_{i=1}^{n}\gamma_i y_i = 0.
\end{aligned}$$

Equivalently, we can write:

$$\begin{aligned}
&\max_{\gamma} \sum_{i=1}^{n}\gamma_i - \frac{1}{2}\sum_{i,j}\gamma_i\gamma_j y_i y_j \mathbf{x}_i \cdot \mathbf{x}_j \\
&\text{s.t. } 0 \leqslant \gamma_i \leqslant \mu, \quad i = 1, \ldots, n, \qquad (5.30) \\
&\sum_{i=1}^{n}\gamma_i y_i = 0,
\end{aligned}$$

with μ as a tuning parameter ($C = 0 \iff \mu = +\infty$). We observe that μ has the opposite interpretation of C, but the same role in the bias-variance trade-off.

At the optimum:

$$\hat{\beta} = \sum_{i=1}^{n}\hat{\gamma}_i y_i \mathbf{x}_i, \tag{5.31}$$

where $\boldsymbol{\beta}$ is a p-dimensional vector and $\boldsymbol{\gamma}$ is an n-dimensional vector. Therefore:

- if $n \gg p$, then the primal problem has fewer parameters, so it is better to solve it rather than the dual problem;
- if $p \gg n$, then the dual problem has fewer parameters, so it is better to solve it rather than the primal problem.

On top of that, many γ_i will be zero. In fact, using the KKT conditions, $\partial_i^* g_i(\boldsymbol{\alpha}^*) = 0$:

$$\hat{\gamma}_i [y_i(\hat{\beta}_0 + \hat{\boldsymbol{\beta}}^\top \mathbf{x}_i) - 1 + \hat{\varepsilon}_i] = 0, \quad i = 1, \dots, n. \tag{5.32}$$

Note. β_0 is estimated from these conditions as an average over n. ○

Take a point in the correct side of the hyperplane. Then:

$$y_i(\hat{\beta}_0 + \hat{\boldsymbol{\beta}}^\top \mathbf{x}_i) > 1 \quad (\text{since } \hat{\varepsilon}_i = 0). \tag{5.33}$$

So, for the product in 5.32 to be zero, we need $\hat{\gamma}_i = 0$. This means that the only non-zero $\hat{\gamma}_i$ are the ones corresponding to the points that lie on the margin or on the wrong side of the hyperplane. These points are still the ones we called *support vectors* earlier, since they are the only ones that contribute to the estimation of $\boldsymbol{\beta}$.

5.3 Support Vector Machines

SVMs extend Support Vector Classifiers to the case of non-linear decision surfaces. There are two main ideas behind this extension:

1. Augment the feature space so that observations become linearly separable in that new space.
2. Can we actually do this without going in a higher dimensional space?

Quadratic decision surface Suppose we have p features $x_1, \dots, x_p$. We can build the augmented feature space in this way:

$$x_1, x_1^2, x_2, x_2^2, \dots, x_p, x_p^2. \tag{5.34}$$

The SVC problem on the augmented space becomes:

$$\begin{aligned} &\min_{\beta_0, \boldsymbol{\beta}, \boldsymbol{\varepsilon}} \frac{1}{2}\|\boldsymbol{\beta}\|^2 \\ &\text{s.t. } y_i\left(\beta_0 + \sum_{j=1}^{p} \beta_{j1} x_{ij} + \sum_{j=1}^{p} \beta_{j2} x_{ij}^2\right) \geqslant 1 - \varepsilon_i, \quad i = 1, \dots, n, \\ &\quad \varepsilon_i \geqslant 0, \quad i = 1, \dots, n, \\ &\quad \sum_{i=1}^{n} \varepsilon_i \leqslant C. \end{aligned} \tag{5.35}$$

In the augmented feature space $\mathbb{R}^{2p}$ we obtain, therefore, a linear decision surface:

$$\mathbf{z} \in \mathbb{R}^{2p} : \beta_0 + \boldsymbol{\beta}^\top \mathbf{z} = 0. \tag{5.36}$$

In the original feature space $\mathbb{R}^p$ we obtain a quadratic decision surface:

$$\mathbf{x} \in \mathbb{R}^p : \beta_0 + \sum_{j=1}^{p} \beta_{j1} x_j + \sum_{j=1}^{p} \beta_{j2} x_j^2 = 0. \tag{5.37}$$

The space can be extended considering interaction terms $x_i x_j$ or even higher degrees, but it becomes computationally expensive.

Recall that in an SVC the decision surface is linear and defined by:

$$\begin{aligned} f(\mathbf{x}) &= \beta_0 + \boldsymbol{\beta}^\intercal \mathbf{x} \\ &= \beta_0 + \sum_{j=1}^{p} \beta_j x_j \\ &= \beta_0 + \boldsymbol{\beta} \cdot \mathbf{x} \\ &= \beta_0 + \sum_{i=1}^{n} \gamma_i y_i \mathbf{x}_i \mathbf{x}. \end{aligned} \tag{5.38}$$

In the optimization (estimation of $\boldsymbol{\gamma}$), we solve:

$$\max_{\boldsymbol{\gamma}} \sum_{i=1}^{n} \gamma_i - \frac{1}{2} \sum_{i,j=1}^{n} \gamma_i \gamma_j y_i y_j \mathbf{x}_i \cdot \mathbf{x}_j. \tag{5.39}$$

We notice that the optimization depends on the data only through the inner products $\mathbf{x}_i \cdot \mathbf{x}_j$. Therefore, it would make sense to replace the inner product with a more general function $K(\mathbf{x}_i, \mathbf{x}_j)$, called *kernel*.

Going higher dimensions Suppose we have a function $\Phi : \mathbb{R}^p \to \mathbb{R}^N$ with $N > p$, that maps the original feature space to a higher-dimensional space. Therefore, we can rewrite $f(\mathbf{x})$ in this space as:

$$f(\mathbf{x}) = \beta_0 + \underbrace{\boldsymbol{\beta}^\intercal}_{\boldsymbol{\beta} \in \mathbb{R}^N} \underbrace{\Phi(\mathbf{x})}_{z} = \beta_0 + \sum_{i=1}^{n} \underbrace{\gamma_i}_{\boldsymbol{\gamma} \in \mathbb{R}^n} y_i \Phi(\mathbf{x}_i) \cdot \Phi(\mathbf{x}). \tag{5.40}$$

So, β_0 and $\boldsymbol{\gamma}$ get estimated by:

$$\max_{\boldsymbol{\gamma}} \sum_{i=1}^{n} \gamma_i - \frac{1}{2} \sum_{i,j=1}^{n} \gamma_i \gamma_j y_i y_j \Phi(\mathbf{x}_i) \cdot \Phi(\mathbf{x}_j). \tag{5.41}$$

Note. We notice that while the βs are N, the γs stay in number n. ○

Since Φ is generally not known, we would not want to compute $\Phi(\mathbf{x}_i) \cdot \Phi(\mathbf{x}_j)$ directly. However, we can rely on the so called *kernel trick*, which consists in finding a function $K : \mathbb{R}^p \times \mathbb{R}^p \to \mathbb{R}$ such that it acts as a dot product in some higher dimensional space for some function Φ. Mercer's theorem states the properties that such function needs to have.

Using such kernel in $f(\mathbf{x})$:

$$f(\mathbf{x}) = \beta_0 + \sum_{i=1}^{n} \gamma_i y_i K(\mathbf{x}_i, \mathbf{x}) \tag{5.42}$$

and the optimization becomes:

$$\max_{\boldsymbol{\gamma}} \sum_{i=1}^{n} \gamma_i - \frac{1}{2} \sum_{i,j=1}^{n} \gamma_i \gamma_j y_i y_j K(\mathbf{x}_i, \mathbf{x}_j). \tag{5.43}$$

The decision surface is, therefore, defined by:

$$\mathbf{x} \colon f(\mathbf{x}) = 0 \tag{5.44}$$

and it is non-linear, with the shape influenced by the choice of the kernel.

The most common choices for the kernel are:

Linear Kernel $$K(\mathbf{x}, \mathbf{z}) = \mathbf{x} \cdot \mathbf{z}. \tag{5.45}$$

Polynomial Kernel $$K(\mathbf{x}, \mathbf{z}) = \left(1 + \sum_{j=1}^{p} \mathbf{x}_j \mathbf{z}_j\right)^n \text{ of degree } n. \tag{5.46}$$

Radial Kernel $$K(\mathbf{x}, \mathbf{z}) = \exp\left(-\gamma \sum_{j=1}^{p} (\mathbf{x}_j - \mathbf{z}_j)^2\right) \text{ with } \gamma > 0. \tag{5.47}$$

If $\gamma = \frac{1}{2\sigma^2}$, then the radial kernel is also called *Gaussian kernel*. For a $\mathbf{z}$ far from $\mathbf{x}$, the kernel gets small: for this, it is also called *local kernel*.

The choice of $\boldsymbol{\gamma}, n, \boldsymbol{\mu}$ is important as it relates to the bias-variance trade-off. Again, CV is needed for the selection of these parameters.

In conclusion, for an SVM we need:

1. A kernel function K with optimized tuning parameters.
2. The calculation of $\binom{n}{2}$ values of $K(\mathbf{x}_i, \mathbf{x}_j)$ to estimate β_0 and $\boldsymbol{\gamma}$.

Multiple classes If there are more than two classes (namely $K > 2$), we can proceed with mainly two approaches:

1. *One vs All*: train K classifiers where the labels are "class c" vs "not class c", for $c = 1, \ldots, K$. One can then combine these classifiers by assigning $\mathbf{x}$ to the class c for which $\left|f_c(\mathbf{x})\right|$ is maximal.

   However, two problems arise with this approach:

   - *Potential ambiguity*: if for a given $\mathbf{x}$ we have $f_c(\mathbf{x}) > 0$ for more than one class, we cannot assign $\mathbf{x}$ to a single class.
   - *Unbalanced data*: if one class is much more represented than the others, the classifier for that class will be more accurate than the others, and it will be more likely to assign $\mathbf{x}$ to that class.

2. *One vs One*: train $\binom{K}{2} = K(K-1)/2$ classifiers, one for each pair of classes. Then, classify $\mathbf{x}$ to the class with the higher number of votes.

   This approach is computationally more expensive, and there could still be potentially ambiguity. However, if K is not too large, this is what is usually done.

SVCs vs Logistic Regression The optimization problem of SVC is, as said before:

$$\begin{aligned} &\min_{\beta_0,\beta,\varepsilon} \frac{1}{2}\|\beta\|^2 \\ &\text{s.t. } y_i(\beta_0+\beta^\top \mathbf{x}_i) \geqslant 1-\varepsilon_i, \quad i=1,\dots,n, \\ &\quad \varepsilon_i \geqslant 0, \quad i=1,\dots,n, \\ &\quad \sum_{i=1}^n \varepsilon_i \leqslant C. \end{aligned} \tag{5.48}$$

The first constraint is equivalent to:

$$\varepsilon_i \geqslant 1-y_i(\beta_0+\beta^\top \mathbf{x}_i). \tag{5.49}$$

If we combine with the second $\varepsilon_i \geqslant 0$, we get:

$$\varepsilon_i \geqslant \max\{0, 1-y_i(\beta_0+\beta^\top \mathbf{x}_i)\} = [1-y_i(\beta_0+\beta^\top \mathbf{x}_i)]_+. \tag{5.50}$$

Using the third constraint:

$$\sum_{i=1}^n [1-y_i(\beta_0+\beta^\top \mathbf{x}_i)]_+ \leqslant \sum_{i=1}^n \varepsilon_i \leqslant C. \tag{5.51}$$

So, the optimization problem can be rewritten as:

$$\begin{aligned} &\min_{\beta_0,\beta} \frac{1}{2}\|\beta\|^2 \\ &\text{s.t. } \sum_{i=1}^n [1-y_i(\beta_0+\beta^\top \mathbf{x}_i)]_+ \leqslant C. \end{aligned} \tag{5.52}$$

Computing the Lagrangian:

$$\mathcal{L}(\beta_0,\beta,\gamma) = \frac{1}{2}\|\beta\|^2 + \gamma\left(\sum_{i=1}^n [1-y_i(\beta_0+\beta^\top \mathbf{x}_i)]_+\right) \tag{5.53}$$

with $\gamma \geqslant 0$ as a tuning parameter, and the equivalent unconstrained optimization problem is:

$$\min_{\beta_0,\beta} \sum_{i=1}^n [1-y_i(\beta_0+\beta^\top \mathbf{x}_i)]_+ + \frac{\lambda}{2}\|\beta\|^2. \tag{5.54}$$

Compared with Ridge:

$$\min_{\beta_0,\beta} \sum_{i=1}^n (y_i-\beta_0-\beta^\top \mathbf{x}_i)^2 + \frac{\lambda}{2}\|\beta\|^2. \tag{5.55}$$

The loss function of SVC is called *hinge loss*, while the one of logistic regression is just the *squared error loss*. So, SVM uses a different loss function:

$$\mathrm{L}(y, f(\mathbf{x})) = \begin{cases} 0 & \text{if } 1-yf(\mathbf{x}) < 0 \iff yf(\mathbf{x}) \geqslant 1, \\ 1-yf(\mathbf{x}) & \text{if } 1-yf(\mathbf{x}) \geqslant 0 \iff yf(\mathbf{x}) < 1. \end{cases} \tag{5.56}$$

We observe that if the point is correctly classified, then the loss is zero. If the point is misclassified, then the loss is proportional to the distance from the decision surface.

For the logistic regression, we don't have a Ridge penalty, but we can add it. The loss function is the negative log-likelihood:

$$\mathrm{L}(y_i, f(\mathbf{x}_i)) = \begin{cases} -\log \mathrm{P}(1 \mid \mathbf{x}_i) & \text{if } y_i = 1, \\ -\log(1 - \mathrm{P}(1 \mid \mathbf{x}_i)) & \text{if } y_i = -1. \end{cases} \tag{5.57}$$

We recall that for logistic regression we have:

$$\mathrm{P}(1 \mid \mathbf{x}) = \frac{e^{\beta_0 + \beta^\top \mathbf{x}}}{1 + e^{\beta_0 + \beta^\top \mathbf{x}}} \tag{5.58}$$

and therefore:

- $y_i = 1$:

$$-\log \mathrm{P}(1 \mid \mathbf{x}_i) = -\log\left(\frac{e^{\beta_0 + \beta^\top \mathbf{x}_i}}{1 + e^{\beta_0 + \beta^\top \mathbf{x}_i}}\right) = \log(1 + e^{-(\beta_0 + \beta^\top \mathbf{x}_i)}). \tag{5.59}$$

- $y_i = -1$:

$$-\log(1 - \mathrm{P}(1 \mid \mathbf{x}_i)) = -\log\left(1 - \frac{e^{\beta_0 + \beta^\top \mathbf{x}_i}}{1 + e^{\beta_0 + \beta^\top \mathbf{x}_i}}\right) = \log(1 + e^{\beta_0 + \beta^\top \mathbf{x}_i}). \tag{5.60}$$

The writing can be simplified by noticing that $y_i \in \{-1, 1\}$:

$$\mathrm{L}(y_i, f(\mathbf{x}_i)) = \log(1 + e^{-y_i f(\mathbf{x}_i)}). \tag{5.61}$$

Let's now compare the two optimization problems:

SVC $\min_{\beta_0, \beta} \sum_{i=1}^{n} [1 - y_i(\beta_0 + \beta^\top \mathbf{x}_i)]_+ + \frac{\lambda}{2}\|\beta\|^2$

LR with Ridge penalty $\min_{\beta_0, \beta} \sum_{i=1}^{n} \log(1 + e^{-y_i(\beta_0 + \beta^\top \mathbf{x}_i)}) + \frac{\lambda}{2}\|\beta\|^2$.

Support Vectors techniques can be extended also to regression problems using the so called *ε-insensitive loss function*:

$$\mathrm{L}(y, f(\mathbf{x})) = \begin{cases} 0 & \text{if } \left|y - f(\mathbf{x})\right| < \varepsilon, \\ \left|y - f(\mathbf{x})\right| - \varepsilon & \text{if } \left|y - f(\mathbf{x})\right| \geqslant \varepsilon. \end{cases} \tag{5.62}$$

that is zero if the residuals are small enough (less than the parameter ε) and proportional to the distance from the decision surface otherwise.

6 Survival analysis

The *survival analysis* is a regression model where the response is given by:

$$T = \text{“Time until an event occurs”}, \tag{6.1}$$

for example, the time until a patient dies, the time until a machine breaks down, etc.

Example. A five-year medical study is conducted in which patients have been treated for cancer. The aim is to predict a patient survival time using features such as patient covariates, type of treatment, etc.

However, some (hopefully many) patients may have survived until the end of the study. These patients have a survival time that is *censored*. All we know is that it is at least five years, but we do not know its value. Other patients may have also dropped out of the study before its end. Survival analysis provides methods for this exact type of data. ◇

There are also many other applications:

1. Modelling churn, *i.e.*, “time until cancellation”.
2. Modelling failure time of a product (reliability studies).
3. The variable can also not be time.
4. Time is naturally right-censored, but we can also have left-censoring, *e.g.*, a pregnancy study where measurements start 250 days after conception and T is the time until the end of pregnancy.
5. Another option is interval-censored data whether an event occurred during a specific period of time, like a week.

Survival curve Let T be:

$$T = \text{“Time until an event occurs”}. \tag{6.2}$$

There are different ways to describe T probabilistically:

$$\underset{\text{PDF}}{f(t) = \mathrm{P}(T = t)}, \qquad \underset{\text{CDF}}{F(t) = \mathrm{P}(T \leqslant t)}, \qquad \underset{\substack{\text{SURVIVAL}\\ \text{CURVE}}}{S(t) = \mathrm{P}(T > t) = 1 - F(t)}. \tag{6.3}$$

Hazard function We define this function as:

$$\begin{aligned} h(t) &= \lim_{\Delta t \to 0} \frac{\mathrm{P}(t < T \leqslant t + \Delta t \mid T > t)}{\Delta t} \\ &= \lim_{\Delta t \to 0} \frac{\mathrm{P}(t < T \leqslant t + \Delta t) \cap \mathrm{P}(T > t)}{\mathrm{P}(T > t)\Delta t} \\ &= \lim_{\Delta t \to 0} \frac{\mathrm{P}(t < T \leqslant t + \Delta t)}{S(t)\Delta t} \\ &= \frac{1}{S(t)} \lim_{\Delta t \to 0} \frac{\mathrm{P}(t < T \leqslant t + \Delta t)}{\Delta t} \\ &= \frac{f(t)}{S(t)}. \end{aligned} \tag{6.4}$$

The functions are all equivalent and can be used for modelling T:

$$f(t) \longleftrightarrow F(t) \longleftrightarrow S(t) \longleftrightarrow h(t). \tag{6.5}$$

6.1 Different models

There are different ways of modelling survival data:

1. Non-parametric.
2. Parametric (assuming a specific shape for the distribution of T).
3. Semi-parametric.

Now we set:

$$\begin{aligned} T &= \text{“Survival time”}, \\ C &= \text{“Censoring time”}. \end{aligned} \tag{6.6}$$

What we observe is:

$$Y\colon\ \min(T, C) = \begin{cases} T & \text{if } T \leqslant C, \\ C & \text{if } T > C. \end{cases} \tag{6.7}$$

In addition, we observe:

$$\delta = \begin{cases} 1 & \text{if } T \leqslant C, \\ 0 & \text{if } T > C. \end{cases} \tag{6.8}$$

that is a binary variable that indicates whether the event of interest occurred or not. For instance, if a person died or left the study before its end.

So, a dataset is given by $\{(Y_i, \delta_i)\}_{i=1}^n$.

6.2 Kaplan-Meier estimator of survival curve

Suppose that we have the following dataset:

$$\{(y_1, 1), (y_2, 0), (y_3, 1), (y_4, 0), (y_5, 1)\}, \tag{6.9}$$

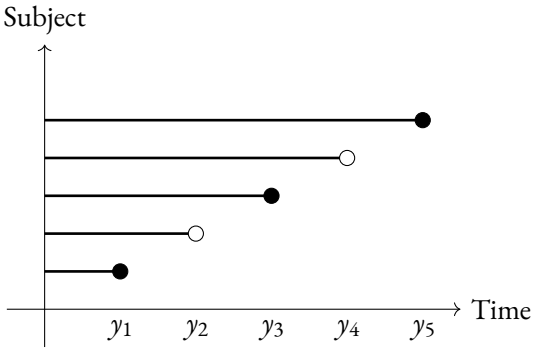

Figure 6.1 *Example of a dataset with $n = 5$ subjects. The circles indicate whether the event of interest occurred (black) or not (white).*

We want to compute, for example:

$$\hat{S}(y_3) = \hat{\mathrm{P}}(T > y_3). \tag{6.10}$$

Some naive options:

- Throw away missing observations:

$$\hat{S}(y_3) = \frac{1}{3}, \tag{6.11}$$

  where we only consider the three subjects for which we are sure that the event of interest occurred (the black circles).

- Consider all observations:

$$\hat{S}(y_3) = \frac{2}{5}, \tag{6.12}$$

  where we consider all the subjects, including those for which we are not sure that the event of interest occurred (the white circles), but we treat them as if they were black circles.

A better option would be to consider only time points at which the event of interest occurred. For example:

$$\hat{S}(y_1) = \hat{\mathrm{P}}(T > y_1) = \frac{4}{5}. \tag{6.13}$$

In this case we account also for patients 2 and 4. To compute $\hat{S}(y_3)$, we can use the following formula:

$$\begin{aligned} \hat{S}(y_3) &= \hat{\mathrm{P}}(T > y_3) \\ &= \mathrm{P}(T > y_3 \mid T \leqslant y_1)\,\mathrm{P}(T \leqslant y_1) + \mathrm{P}(T > y_3 \mid T > y_1)\,\mathrm{P}(T > y_1) \\ &= \hat{\mathrm{P}}(T > y_3 \mid T > y_1)\hat{\mathrm{P}}(T > y_1) \\ &= \frac{2}{3} \cdot \frac{4}{5} = \frac{8}{15}. \end{aligned} \tag{6.14}$$

We used the fact that $\mathrm{P}(T > y_3 \mid T \leqslant y_1) = 0$ since $y_3 > y_1$. Obviously, we can also compute $\hat{S}(y_5)$:

$$\hat{S}(y_5) = \hat{\mathrm{P}}(T > y_5) = 0, \tag{6.15}$$

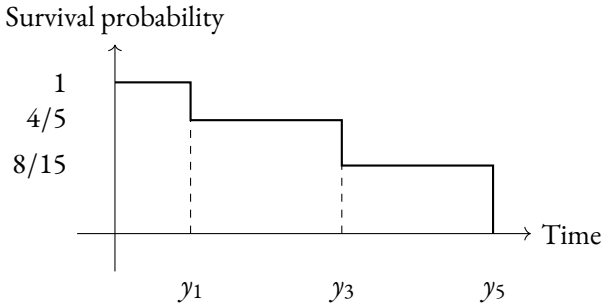

Figure 6.2 *Kaplan-Meier estimator of the survival curve.*

since there is no data point greater than y_5. The survival curve is, therefore, given by the step function showed in Figure 6.2.

Usually, the time points are way more and the survival curve is smoother. Also, some observations reach the end of the study, so the survival curve does not go to zero on the right.

Non-parametric approach More formally, let a generic dataset be:

$$d_1, \dots, d_k: \text{ death times}, \tag{6.16}$$

Let r_k be:

$$r_k = \text{“Number of patients alive and in the study just before } d_k\text{”}. \tag{6.17}$$

This is also called the *risk set* at time d_k. Let q_k be:

$$q_k = \text{“Number of patients that died at time } d_k\text{”}. \tag{6.18}$$

Then:

$$\begin{aligned} S(d_k) &= \mathrm{P}(T > d_k \mid T > d_{k-1}) \underbrace{\mathrm{P}(T > d_{k-1})}_{S(d_{k-1})} \\ &= \prod_{j=1}^{k} \mathrm{P}(T > d_j \mid T > d_{j-1}) \end{aligned} \tag{6.19}$$

is estimated using:

$$\hat{\mathrm{P}}(T > d_j \mid T > d_{j-1}) = \frac{r_j - q_j}{r_j}, \tag{6.20}$$

so that:

$$\hat{S}(d_k) = \prod_{j=1}^{k} \frac{r_j - q_j}{r_j}, \quad k = 1, \dots, K, \tag{6.21}$$

and this is called the *Kaplan-Meier estimator* of the survival curve (non-parametric estimator). For times (d_k, d_{k+1}) we set $\hat{S}(t) = \hat{S}(d_k)$, so that the survival curve is a step function.

Observation. This is similar to computing a histogram at the level of survival curves and considering missing data. △

Parametric approach We can assume that the distribution of T has a certain parametric form. Then, $f(t)$, $F(t)$, $S(t)$ and $h(t)$ all depend on parameters.

Given n observations of the form (y_i, δ_i), we compute the likelihood as:

$$\begin{aligned} \mathrm{L} &= \prod_{i=1}^{n} f(y_i)^{\delta_i} S(y_i)^{1-\delta_i} \\ &= \prod_{i=1}^{n} \left(\frac{f(y_i)}{S(y_i)}\right)^{\delta_i} S(y_i) \\ &= \prod_{i=1}^{n} h(y_i)^{\delta_i} S(y_i). \end{aligned} \tag{6.22}$$

If we assume that T is exponentially distributed, then:

$$\begin{aligned} f(t) &= \lambda e^{-\lambda t}, \\ F(t) &= 1 - e^{-\lambda t}, \\ S(t) &= 1 - F(t) = e^{-\lambda t}, \\ h(t) &= \frac{f(t)}{S(t)} = \lambda. \end{aligned} \tag{6.23}$$

Then, λ is estimated via maximum likelihood:

$$\mathrm{L} = \prod_{i=1}^{n} \lambda^{\delta_i} e^{-\lambda y_i} \implies \hat{\lambda} = \operatorname*{argmax}_{\lambda} \mathrm{L} \tag{6.24}$$

and the survival curve is estimated as:

$$\hat{S}(t) = e^{-\hat{\lambda} t}. \tag{6.25}$$

Regression approaches Here we want to predict T from a number of covariates $X_1, \ldots, X_p$. Non-parametric approaches can be used in this case as well, but they require some stratification of the observations into groups, which is not really feasible for a large number of predictors. With one predictor, *e.g.* X = "Gender", one could split data into males and females and compute a Kaplan-Meier curve for each group.

After having computed the curves, we can conduct a *log-rank test* to determine if differences between the two are statistically significant.

It is easier to account for covariates under a parametric approach. For example, is one assume an exponential distribution, we could make this assumption conditional on $\mathbf{x}$, like in GLMs:

$$T \mid \mathbf{X} = \mathbf{x} \sim \mathrm{Exp}(\lambda(\mathbf{x})), \tag{6.26}$$

with $\log(\lambda(\mathbf{x})) = \beta_0 + \boldsymbol{\beta}^\top \mathbf{x}$. In particular:

$$\log(\lambda(\mathbf{x}_i)) = \log(h(t \mid \mathbf{x}_i)) = \beta_0 + \boldsymbol{\beta}^\top \mathbf{x}_i, \quad i = 1, \ldots, n. \tag{6.27}$$

The likelihood now depends on β_0 and $\boldsymbol{\beta}$:

$$\hat{\lambda}(\mathbf{x}) = \exp(\hat{\beta}_0 + \hat{\boldsymbol{\beta}}^\top \mathbf{x}) \tag{6.28}$$

and we have a survival curve for each realization $\mathbf{x}$ of the covariates:

$$\hat{S}(t \mid \mathbf{x}) = \exp(-\hat{\lambda}(\mathbf{x})t). \tag{6.29}$$

The curve is, therefore, typically plotted by fixing all covariates to a level (for example, mean, mode, etc.) and changing one variable only (x_j) to see how the curve changes with respect to it.

Note. The exponential may be too restrictive in many applications. Weibull distribution (two parameters) is quite common in engineering (reliability modelling). ○

6.3 Cox-Proportional hazard model

In this semi-parametric model, the hazard function is given by:

$$h(t \mid \mathbf{x}_i) = h_0(t) \exp(\mathbf{x}_i^\intercal \beta), \tag{6.30}$$

or equivalently:

$$\log h(t \mid \mathbf{x}_i) = \log h_0(t) + \mathbf{x}_i^\intercal \beta. \tag{6.31}$$

Basically, it replaces the intercept with an arbitrary function of time, and considers β without the intercept, *i.e.*:

$$\beta = \begin{pmatrix} \beta_1 \\ \vdots \\ \beta_p \end{pmatrix}. \tag{6.32}$$

It is *semi-parametric* because a part is parametric (the β) and a part is non-parametric (the $h_0(t)$).

We notice that, in this way, the hazard is not constant in time. The non-parametric part $h_0(t)$ is called *baseline hazard* since:

$$h_0(t) = h(t \mid x_1 = 0, \dots, x_p = 0). \tag{6.33}$$

A generic individual with covariates $\mathbf{x}_i$ has hazard that is a multiple of the baseline hazard, and the multiplicative factor is $\exp(\mathbf{x}_i^\intercal \beta)$.

Note. Why is it called proportional hazard? Consider one predictor:

$$x = \begin{cases} 0 & \text{female,} \\ 1 & \text{male.} \end{cases} \tag{6.34}$$

Then:

$$\begin{aligned} &h(t \mid x = 0) = h_0(t), \\ &h(t \mid x = 1) = h_0(t) \exp(\beta_1) \iff \log h(t \mid x = 1) = \log h_0(t) + \beta_1. \end{aligned} \tag{6.35}$$

Since $h(0)$ can be flexible, this is a less restrictive model. Moreover, since we have a multiplicative factor, two survival curves for different covariate values cannot cross. ○

Interpretation of the coefficients The quantity:

$$\exp(\beta^\intercal \mathbf{x}_i) \tag{6.36}$$

is the global relative risk for patient i. Individual βs capture the effect of each covariate on the hazard, keeping all the other covariates at a constant level. In particular, we can distinguish between the following cases:

- $\beta_j > 0 \implies e^{\beta_j} > 1$: increased hazard of dying at any time t given that you have survived up to time t as x_j increases by one unit.
- $\beta_j < 0 \implies e^{\beta_j} < 1$: decreased hazard of dying at any time t given that you have survived up to time t as x_j increases by one unit.

6.4 Partial likelihood

We said that $h_0(t)$ is arbitrary, in the sense that it is unknown. Therefore, the likelihood function cannot be evaluated and then optimized. David Cox developed a novel inference procedure by approximating the full likelihood via a partial likelihood that does not depend on $h_0(t)$.

We can look at the data generative process in a different way:

$$\text{event} = (t, \text{patient}). \tag{6.37}$$

The full likelihood can be written as:

$$\prod_{k=1}^{K} g(t_k) f(\text{patient}_k \mid t_k). \tag{6.38}$$

The $g(t_k)$ term represent something that happened at time t_k and the $f(\text{patient}_k \mid t_k)$ term represent the probability that the patient k died at time t_k conditioned on the fact that something happened at time t_k.

The idea behind the partial likelihood is to ignore the $g(t_k)$ term and to focus on the part of the likelihood that most depend on β:

$$\begin{aligned} \text{PL}(\beta) &= \prod_{k=1}^{K} f(\text{patient}_k \mid t_k) \\ &= \prod_{k=1}^{K} \frac{h_0(t_k) \exp(\beta^\intercal \mathbf{x}_k)}{\sum_{j \in R(t_k)} h_0(t_k) \exp(\beta^\intercal \mathbf{x}_j)} \\ &= \prod_{k=1}^{K} \frac{\exp(\beta^\intercal \mathbf{x}_k)}{\sum_{j \in R(t_k)} \exp(\beta^\intercal \mathbf{x}_j)} = f(\beta), \end{aligned} \tag{6.39}$$

where $R(t_k)$ is the risk set at time t_k, *i.e.* the set of patients that are still alive at time t_k. This is a multinomial likelihood. We notice some facts:

- $\text{PL}(\beta)$ does not depend on $h_0(t)$.
- Find β that maximizes $\text{PL}(\beta)$ is equivalent to find β that maximizes the full likelihood.
- $h_0(t)$ can be estimated a posteriori (given $\hat{\beta}$).

6.5 Survival models for prediction

Relative risk can be calculated for any individual on some test set:

$$\hat{\eta}_i = \hat{\beta}_1 x_{i1} + \cdots + \hat{\beta}_p x_{ip}, \quad i = 1, \ldots, n. \tag{6.40}$$

We also need to stratify the patients into risk groups, for example, `low`, `medium` and `high` risk. Then, we can check for covariates that are particularly prevalent in each group.

Since it holds:

$$\hat{\eta}_{i^*} > \hat{\eta}_i \implies t_i > t_{i^*}, \tag{6.41}$$

we could use

$$C = \frac{\sum_{(i,\, i^*) \text{ s.t. } y_i > y_{i^*}} \mathbb{1}(\hat{\eta}_{i^*} > \hat{\eta}_i)\delta_{i^*}}{\sum_{(i,\, i^*) \text{ s.t. } y_i > y_{i^*}} \delta_{i^*}} \tag{6.42}$$

as a measure of the predictive performance of the model, with $\hat{\boldsymbol{\beta}}$ estimated from training data. We multiply by δ_{i^*} to select only those i^* that are not censored.

Basically, with this ratio we are checking how many pairs of patients are correctly ordered by the model given the estimated relative risk. We compare the predicted order (through the relative risk $\hat{\eta}_i$) with the true order (through the observed survival times y_i).

6.6 Application to dynamic networks

Definition 3 (Relational event). *A* relational event *is an instantaneous interaction between two or more actors at a particular point in time.*

Example. Some examples are the sending of emails, likes, messages, bike sharing, etc. ◇

The generative process of a relational event produces a sequence of events:

$$\text{event} = (t, \text{sender}, \text{receiver}), \tag{6.43}$$

with sender $\in S$ and receiver $\in R$. We have two cases for S and R sets:

- $S = R$: the sender and the receiver are the same set of actors, for example, in email communication.
- $S \cap R = \emptyset$: as in two-mode networks, the sender and the receiver are different sets of actors.

Relational event model Modelling hazard of interactions:

$$h(t \mid \mathbf{x}_{s,r}) = h_0(t) \exp(\boldsymbol{\beta}^\top \mathbf{x}_{s,r}(t)), \tag{6.44}$$

where $\mathbf{x}_{s,r}$ is a vector of covariates associated to the pair (s, r) at time t. The aim is to infer $\boldsymbol{\beta}$ from data.

Examples. Some examples of covariates are:

- Age difference between sender and receiver in a friendship network.
- Physical distance between sender and receiver.
- Whether sender is a teacher or a student (node-level).

- Reciprocity: whether (r, s) happened before time t.
- Triadic (transitive) closure:

$$x_{\mathrm{sr}}(t) = \begin{cases} 1 & \text{if } (s,k) \text{ and } (k,r) \text{ happened before time } t, \\ 0 & \text{otherwise.} \end{cases} \tag{6.45}$$

The first two are called *dyadic*. The first three are exogenous, while the last two are endogenous. ◇

Let's analyze the partial likelihood in this context. We end up with a similar expression as before:

$$\mathrm{PL}(\beta) = \prod_{k=1}^{K} \frac{\exp(\beta^{\mathsf{T}} \mathbf{x}_{s_k, r_k}(t_k))}{\sum_{(s_{k^*}, r_{k^*}) \in R(t_k)} \exp(\beta^{\mathsf{T}} \mathbf{x}_{s_{k^*}, r_{k^*}}(t_k))}. \tag{6.46}$$

$R(t_k)$ has about p^2 elements with p as the number of units. Computationally more efficient methods are developed considering this new partial likelihood.

7 Unsupervised learning

Up to now we only discussed *supervised learning* methods, for which we have data related both to $\mathbf{X}$ and Y. Our goal was to learn a rule to predict Y from $\mathbf{X}$. In *unsupervised learning* methods, instead, we only have data on $\mathbf{X}$. Here, the objective is to find latent structures and patterns in the data through a form of exploratory data analysis. These methods are also more subjective than supervised learning methods and can in fact lead to a supervised learning procedure.

Some examples of unsupervised learning methods are:

- We can search for patterns in residuals: if we see a structure, it's probable we missed some important predictor in the model.
- Principal Component Regression: we can use the principal components of $\mathbf{X}$ as predictors in a regression model.

The two main approaches to unsupervised learning are:

1. Clustering
2. Principal Component Analysis (PCA)

7.1 Clustering

The goal of clustering is to describe the data structure via a partition into subgroups of observations. In contrast to classification, where we know the groups, here we need to deduce them from the data.

Formally, we split data D into K clusters, $C_1, C_2, \ldots, C_K$ that form a partition, *i.e.*:

$$\bigcup_{k=1}^{K} C_k = D, \qquad C_i \cap C_j = \emptyset, \quad i \neq j, \tag{7.1}$$

into meaningful (homogeneous) clusters. We can characterize homogeneity in different ways, such as:

- Similarity within groups,
- Dissimilarity between groups.

The two main ingredients for this method are:

1. The choice of an *optimality criterion* to decide the best partition.
2. Devising an *optimization procedure*, in which the optimality criterion is used.

Optimality criterion It is typically defined based on a *distance measure*, *i.e.*, observations in one cluster should be closer to each other according to this measure. A popular choice is the *Euclidean distance*.

In general:

$$C_1, \dots, C_K = \underset{\substack{\Gamma_1,\dots,\Gamma_K \\ \text{partition of } D}}{\operatorname{argmin}} \sum_{i=1}^{K} W(\Gamma_i), \tag{7.2}$$

with $W(\Gamma_i)$ the sum of distances of any two objects in Γ_i. In the case of the Euclidean distance, we have:

$$W(\Gamma_i) = \frac{1}{2|\Gamma_i|} \sum_{\mathbf{x},\mathbf{y} \in \Gamma_i} \sum_{l=1}^{p} (x_l - y_l)^2. \tag{7.3}$$

This is also referred to as *within-cluster variation*, since the W is equivalent to:

$$W(\Gamma_i) = \sum_{\mathbf{x} \in \Gamma_i} \sum_{l=1}^{p} (x_l - \bar{x}_{il})^2, \quad \text{with } \bar{x}_{il} = \frac{1}{|\Gamma_i|} \sum_{\mathbf{x} \in \Gamma_i} x_l. \tag{7.4}$$

The $\bar{\mathbf{x}}_{il}$ is the sample mean of predictor l in cluster Γ_i.

Proof. Let's consider the case $p = 1$. From Equation (7.4) we have:

$$\begin{aligned}
2W(\Gamma_i) &= \frac{1}{|\Gamma_i|} \sum_{x,\, y \in \Gamma_i} (x - y)^2 \\
&= \frac{1}{|\Gamma_i|} \sum_{x \in \Gamma_i} \sum_{y \in \Gamma_i} (x^2 - 2xy + y^2) \\
&= \frac{1}{|\Gamma_i|} \left(|\Gamma_i| \sum_{x \in \Gamma_i} x^2 - 2 \sum_{x \in \Gamma_i} x \underbrace{\sum_{y \in \Gamma_i} y}_{|\Gamma_i|\bar{x}_i} + |\Gamma_i| \underbrace{\sum_{y \in \Gamma_i} y^2}_{y=x} \right) \\
&= 2 \sum_{x \in \Gamma_i} x^2 - 2|\Gamma_i|\bar{x}_i \sum_{x \in \Gamma_i} x \\
&= 2 \sum_{x \in \Gamma_i} x^2 - 2|\Gamma_i|^2 \bar{x}_i^2 \\
&= 2 \sum_{x \in \Gamma_i} (x - \bar{x}_i)^2.
\end{aligned} \tag{7.5}$$

□

Another formulation for the optimality criterion is the so called *between-cluster variation*, that comes from the idea that minimizing the within-cluster variation is equivalent to maximizing the between-cluster variation, *i.e.*, dissimilarity between clusters. Indeed, the overall distance between any pair of observations is a constant that can be split into the sum of the within-cluster variation and the between-cluster variation. Therefore, minimizing one is equivalent to maximizing the other.

Optimization Algorithm The optimal problem is actually NP-hard, since there are approximately K^n ways to split n observations into K clusters. Therefore, efficient greedy approaches are needed. However, they will only lead to local optima. Popular choices are:

K-means This method uses the Euclidean distance as the optimality criterion and fixes K. The algorithm is the following:

1. *Random initialization*: assign a random cluster to each observation. This can be done *uniformly* (typically used) or at random.

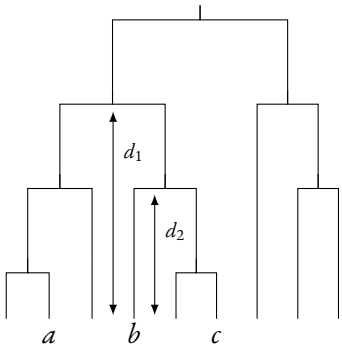

FIGURE 7.1 *Example of a dendogram. a, b and c are three observations.*

2. *Iterate*:
   - For each cluster $i = 1, \dots, K$ compute the centroid $\bar{\mathbf{x}}_i$ of the cluster.
   - Assign each observation $1, \dots, n$ to the cluster whose centroid is closest to it, with respect to the Euclidean distance.

There is no guarantee of a global maximum, so a good practice is to perform the random initialization a certain number of times and keep the best result.

HIERARCHICAL CLUSTERING It is a family of clustering techniques with various distance measures that can be used. Differently to K-means, K is not fixed a priori. The output is a hierarchy of nested clusters, with the hierarchy represented as a *dendogram* (FIGURE 7.1). In the figure, similarity is pictured vertically and not horizontally, in the sense that the observation b and c are "closer" to each other than a and b, because the split happened at a higher level of the hierarchy ($d_1 > d_2$ in the figure).

In addition to the distance between the observations, the method needs a distance between clusters. This is called *linkage criterion* and a common choice for it is the *average linkage*:

$$d_{\text{AL}(C_i,C_j)} = \frac{1}{|C_i||C_j|} \sum_{\mathbf{x},\mathbf{y} \in C_i} d(\mathbf{x}, \mathbf{y}), \tag{7.6}$$

where $d(\mathbf{x}, \mathbf{y})$ is the distance between two observations, such as the Euclidean distance.

The algorithm can proceed in a bottom-up or top-down approach:

- *Agglomerative*: start with each observation in one cluster and then merge the two clusters that are closest to each other; continue until everything is in one cluster.
- *Divisive*: start with everything in one cluster and perform recursive splitting of clusters according to the optimality criterion.

Selection of $\mathbb{K}$ To select K, we can use the *elbow method*, which consists in plotting the optimality criterion as a function of K and looking for an elbow in the plot (FIGURE 7.2). The idea is that, as K increases, the optimality criterion will decrease, but after a certain point the improvement will be negligible. Therefore, we look for the point where the improvement starts to be negligible, which is exactly the elbow of the plot.

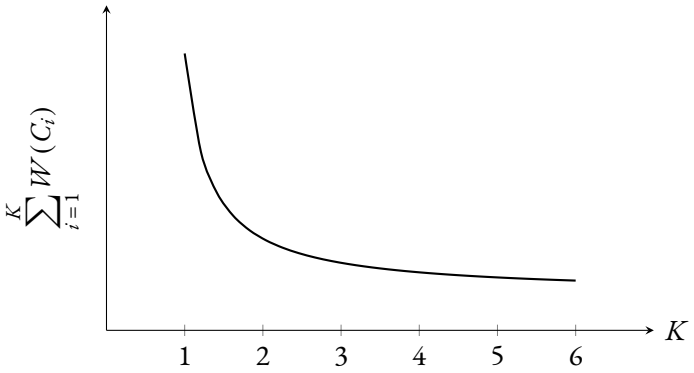

Figure 7.2 *Example of the curve of the elbow method for selecting K. In this case, the elbow is at $K = 2$.*

7.2 Clustering Network Data

This is a special case of clustering, where the data are represented as a graph, with nodes and edges. It is also called *community detection*. The goal is, given a network made of nodes (actors) and edges (interactions), to find unknown groupings of nodes.

We have three main approaches to this problem:

- Hierarchical clustering,
- Spectral clustering (K-means),
- Stochastic block models (mixture models).

Optimality criterion To define an optimality criterion we need to understand what groups have in common.

Let $\ell = (C_1, \ldots, C_K)$ be a partition of the nodes and $f_{ij}(\ell)$ the fraction of edges in the network that connect a node in C_i to a node in C_j. In particular, $f_{ii}(\ell)$ is the fraction of edges in cluster C_i. We can define the following optimality criterion (*modularity*) as:

$$M(\ell) = \sum_{i=1}^{K} (f_{ii}(\ell) - f_{ii}^*)^2, \tag{7.7}$$

where f_{ii}^* is the expected value of $f_{ii}(\ell)$ under a random assignment of edges to nodes. Large values of modularity suggest *non-trivial* (*i.e.*, not random) group structures.

Hierarchical clustering We can apply the hierarchical clustering method, so we merge/split nodes/clusters that maximize the modularity.